

Indice

6	Interfaccia seriale	1
6.1	Esercizio 10	1
6.1.1	Esercizio 10.1	1
6.1.2	Esercizio 10bis	27
7	SWITCH MULTISTADIO	33
7.1	Esercizio 11	33
7.1.1	Esercizio 11.1	33
7.1.2	Esercizio 11.2	45
7.1.3	Esercizio 11.3	57
8	Esercizio 12	78
8.0.1	Progetto e Architettura	79
8.0.2	Implementazione	80
8.0.3	Simulazione	96
9	Timing Analysis	107

Capitolo 6

Interfaccia seriale

6.1 Esercizio 10

6.1.1 Esercizio 10.1

Partendo dall'implementazione fornita dalla Digilent di un dispositivo UART-RS232 (componente RS232RefComp.vhd), progettare, implementare e simulare in VHDL un sistema composto da 2 unità A e B che condividono lo stesso segnale di clock e comunicano tra loro mediante interfaccia seriale. Il sistema A contiene una ROM di 8 locazioni da 1 byte ciascuno, un contatore `CONT_A` per scandire le locazioni della ROM e una `UART_A`, mentre il sistema B contiene una memoria MEM di 8 locazioni da 1 byte ciascuno, un contatore `CONT_B` per scandire le locazioni della MEM e una `UART_B`. Quando un segnale `WR` viene asserito nell'unità A, viene prelevato un byte dalla ROM e inviato all'unità B, che dovrà riceverlo e salvarlo in MEM.

Progetto e architettura

Come richiesto dalla traccia abbiamo implementato due nodi: A, che fa da trasmettitore, B che fa da ricevitore. Successivamente i due nodi vengono connessi tra loro mediante il protocollo "RS232", quest'ultimo contiene l'implementazione dell'UART configurabile in trasmissione o ricezione. L'UART è un dispositivo hardware che supporta la comunicazione seriale sincrona e asincrona. Nell'implementazione a noi fornita troviamo sia il controller della ricezione che della trasmissione, in particolar modo il ricevitore gestisce i seguenti segnali di input:

- CLK;
- RST;
- RXD: segnale dato ricevuto;
- RDA: flag che segnala quando un frame è stato completamente ricevuto;
- RD: segnale di controllo per segnalare la lettura del dato dal dispositivo.

Invece, i segnali di output sono i seguenti:

- DBOUT: uscita parallela UART in ricezione;
- RDA;
- PE: flag per determinare se il dato è correttamente ricevuto;

- OE: flag per verificare se c'è stato un errore di overrun (il sistema ricevente non è in grado di elaborare un dato in arrivo prima che ne sopraggiunga un altro);
- FE: flag per segnalare che non è presente il bit di stop.

Il trasmettitore presenta come segnali di ingresso i seguenti:

- CLK;
- RST;
- DBIN: ingresso dato parallelo;
- WR: segnale di abilitazione alla scrittura;

I segnali di output invece sono:

- TXD: segnale dato trasmesso serialmente;
- TBE: flag che segnala che il dato è stato caricato nello shift register, inizializzato a '1' per la trasmissione.

A partire dalla configurazione fornita, abbiamo implementato la seguente architettura: ARCHITETTURA MANCANTE

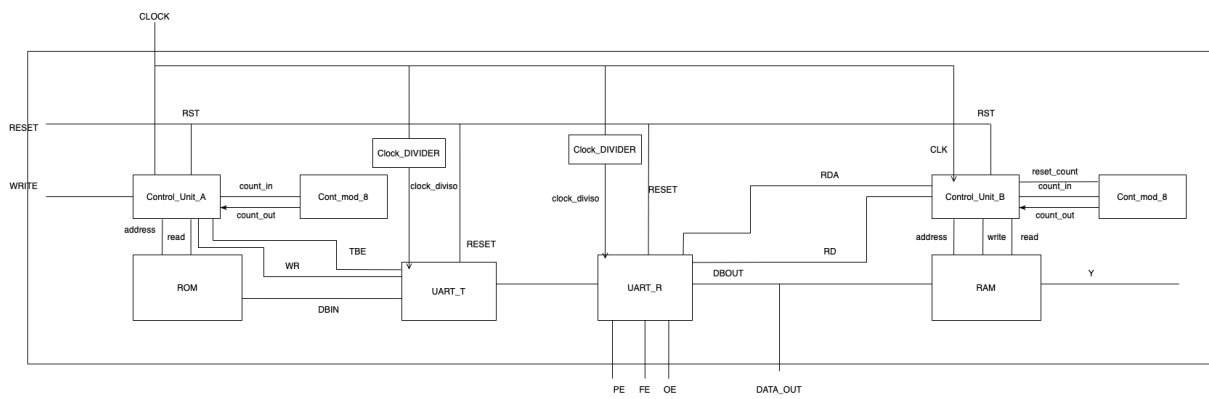


Figura 6.1: Architettura 10

All'interno dell'architettura sono presenti elementi già trattati nella tesi-
na come: contatore, memoria, ROM. Di nostro interesse è stato sviluppare
i nodi A e B. Il nodo A, che funge da trasmettitore, dovrà trasmettere gli
elementi contenuti nella ROM e fornirli alla UART. La logica è visibile dal
seguito automa:

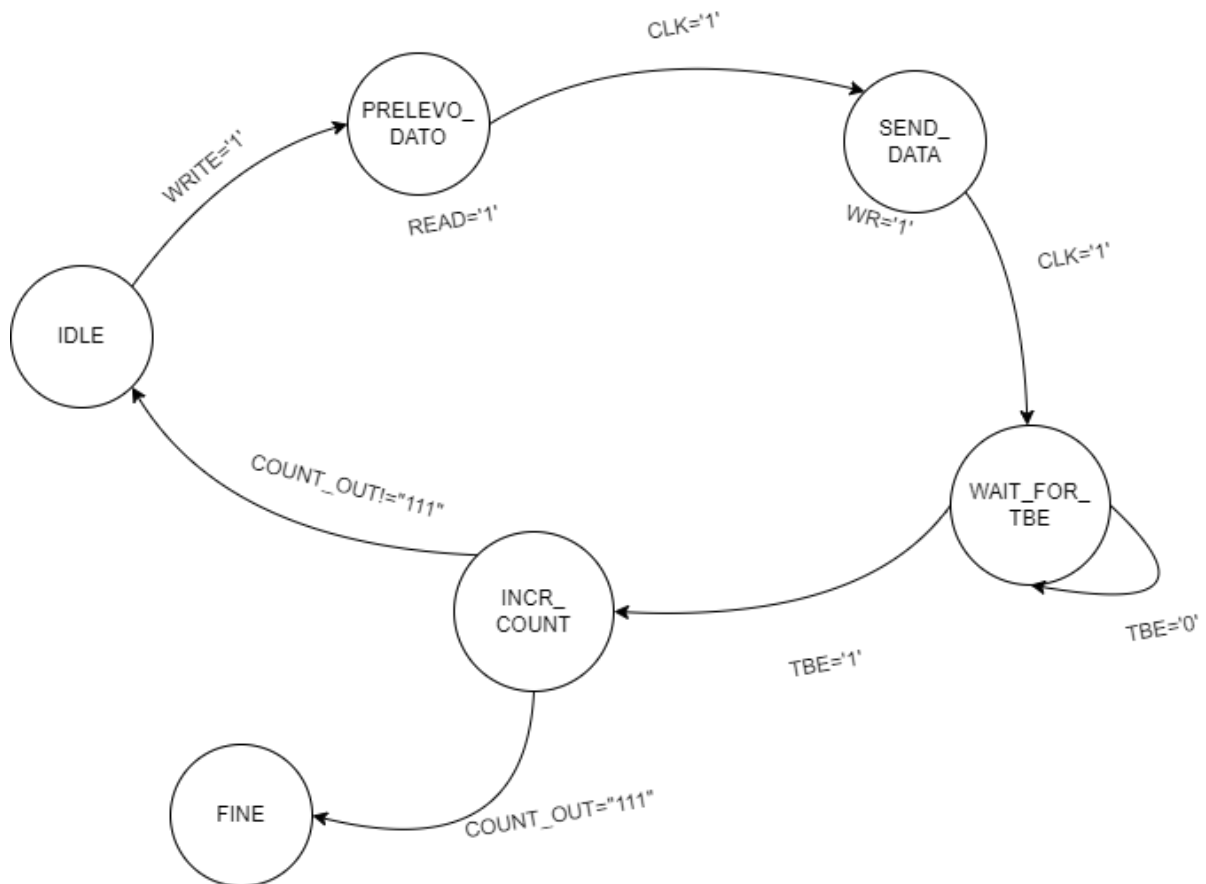


Figura 6.2: Automa 10 nodo A

Il nodo B funge da ricevitore, riceverà i dati tramite UART, e li memo-
rizzerà all'interno della sua memoria. La logica è sintetizzata dal seguente
automa:

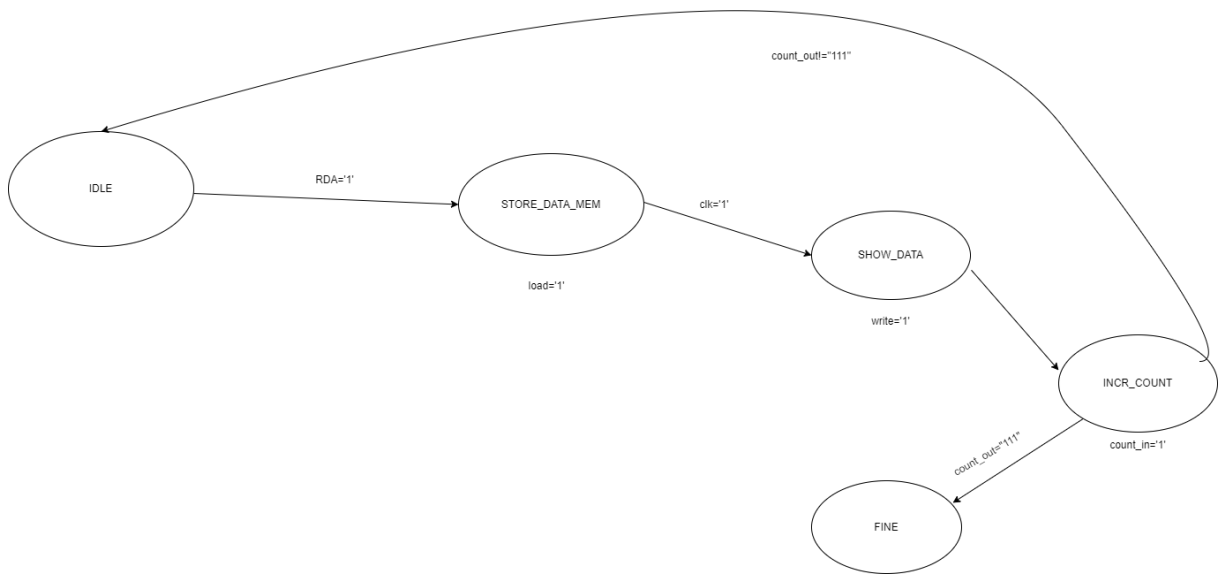


Figura 6.3: Automa 10 nodo B

Implementazione

Implementazione Control Unit A:

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5 entity Control_unit_A is
6   Port (
7     clock: in std_logic;
8     reset : in std_logic;
9     start: in std_logic;
10    TBE: in std_logic;
11    count_out: in std_logic_vector(2 downto 0);
12    count_in :out std_logic;
13    address: out std_logic_vector(2 downto 0);

```

```
14     read: out std_logic;
15     WR: out std_logic
16
17 );
18 end Control_unit_A;
19
20 architecture Behavioral of Control_unit_A is
21
22     type state is (IDLE, PRELEVO_DATO, SEND_DATA, WAIT_FOR_TBE,
23                   INCR_COUNT, FINE);
24
25     signal current_state, next_state: state;
26
27 begin
28     reg_stato: process(clock, reset)
29     begin
30         if(clock'event and clock='1') then
31             if(reset='1') then
32                 count_out<= (others=>'0');
33                 current_state <=idle;
34             else
35                 current_state <=next_state;
36             end if;
37         end if;
38     end process;
39
40     comb: process(current_state, start, count_out, TBE)
41     begin
42
43         read<='0';
```

```
41      WR<='0';
42      count_in<='0';
43      address<=count_out;
44
45
46
47  CASE current_state is
48
49      WHEN IDLE =>
50
51
52          if(start='1') then
53              next_state <= PRELEVO_DATO;
54
55          else
56              next_state <= IDLE;
57          end if;
58
59      --fornisce i segnali di caricamento operandi
60      WHEN PRELEVO_DATO=>
61
62          read<='1';
63
64          --address<=count_out;
65          next_state <= SEND_DATA;
66
67      --acquisisce gli operandi, su cui il sommatore inizia
        a lavorare immediatamente
```

```
68      WHEN SEND_DATA=>
69
70          WR<='1';
71          next_state<=WAIT_FOR_TBE;
72
73
74      WHEN WAIT_FOR_TBE=>
75          if TBE='1' then
76              next_state<= INCR_COUNT;
77          else
78              next_state<=WAIT_FOR_TBE;
79          end if;
80
81      WHEN INCR_COUNT =>
82          count_in<='1';
83
84          if count_out="111" then
85              next_state<=FINE;
86
87          else
88              next_state<= IDLE;
89
90          end if;
91
92
93      WHEN FINE =>
94
95          next_state<= FINE;
```

```
96
97
98
99     end CASE;
100 end process;
101
102
103
104 end Behavioral;
```

Implementazione Nodo A:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6  entity A is
7      Port (
8          CLOCK: in std_logic;--master clock
9          RESET: in std_logic;--reset
10         WRITE: in std_logic;-- segnale per l avvio dell
            operazione di lettura da rom
11         TBE: in std_logic; --uscita derlla uart in trasmissione
12         WR: out std_logic;--segnale da connettere alla uart
13         data_out: out std_logic_vector(7 downto 0)
14
15     );
16 end A;
17
```

```
18 architecture Behavioral of A is
19 component Control_unit_A is
20     Port (
21         clock: in std_logic;
22         reset : in std_logic;
23         start: in std_logic;
24         TBE: in std_logic;
25         count_out: in std_logic_vector(2 downto 0);
26         count_in :out std_logic;
27         address: out std_logic_vector(2 downto 0);
28         read: out std_logic;
29         WR: out std_logic
30     );
31 end component;
32
33
34
35 component cont_mod8 is
36     generic(
37         Log: integer := 2
38     );
39
40     port( clock, reset: in std_logic;
41         count_in: in std_logic;
42         count: out std_logic_vector(Log-1 downto 0));
43 end component;
44
45 component ROM is
```

```
46  generic (
47      Number_of_bit: integer:=8;
48      Number_of_locations: integer :=8;
49      Log: integer :=3 --logaritmo in base due di
        number_of_location
50  );
51
52  Port (
53      read: in std_logic;
54      address :in std_logic_vector(Log-1 downto 0);
55      dout: out std_logic_vector (Number_of_locations-1
        downto 0)
56  );
57  end component;
58
59  signal count_out,address: std_logic_vector(2 downto 0):=(
        others=>'0');
60  signal count_in,read: std_logic:='0';
61  begin
62
63  CU_A: Control_unit_A port map(CLOCK,RESET,WRITE,TBE,count_out,
        count_in,address,read,WR);
64  CONTATORE_A: cont_mod8 generic map(3) port map (CLOCK,RESET,
        count_in,count_out);
65  MEMORIA_ROM: ROM generic map (8,8,3) port map(read,address,
        data_out);
66
67
```

```
68 end Behavioral;
```

Implementazione Control Unit B:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4
5  entity Control_Unit_B is
6      Port (
7          clock: in std_logic;
8          reset: in std_logic;
9          count_out : in std_logic_vector(2 downto 0);
10         RDA: in std_logic;
11         RD: out std_logic;
12         count_in :out std_logic;
13         address: out std_logic_vector(2 downto 0);
14         load: out std_logic;
15         write_mem: out std_logic
16
17     );
18 end Control_Unit_B;
19
20 architecture Behavioral of Control_Unit_B is
21
22     type state is (IDLE, STORE_DATA_IN_MEM, SHOW_DATA, INCR_COUNT,
23                   FINE);
24
25     signal current_state,next_state: state;
```

```
26 begin
27
28 reg_stato: process(clock,reset)
29     begin
30         if(clock'event and clock='1') then
31             if(reset='1') then
32                 count_out<= (others=>'0');
33                 current_state <=idle;
34             else
35                 current_state <=next_state;
36             end if;
37         end if;
38     end process;
39
40 comb: process(current_state,RDA,count_out)
41 begin
42     write_mem<='0';
43     load<='0';
44     count_in<='0';
45     address<=count_out;
46     RD<='0';
47
48
49 CASE current_state is
50
51     WHEN IDLE =>
```

```
54         if (RDA='1') then
55             next_state <= STORE_DATA_IN_MEM;
56
57         else
58             next_state <= IDLE;
59         end if;
60
61         --fornisce i segnali di caricamento operandi
62         WHEN STORE_DATA_IN_MEM=>
63
64
65             RD<='1';
66             load<='1';
67             next_state <= SHOW_DATA;
68
69         --acquisisce gli operandi, su cui il sommatore inizia
70             a lavorare immediatamente
71         WHEN SHOW_DATA=>
72
73
74
75             write_mem<='1';
76
77
78
79             next_state<= INCR_COUNT;
80
81
82
83         WHEN INCR_COUNT =>
84             count_in<='1';
```

```
81         if count_out="111" then
82             next_state<=FINE;
83         else
84             next_state<= IDLE;
85         end if;
86
87
88
89
90         WHEN FINE =>
91
92             next_state<= FINE;
93
94
95         end CASE;
96     end process;
97
98
99 end Behavioral;
```

Implementazione nodo B:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6 entity B is
7     Port (
8         clock: in std_logic;
```

```

9      reset: in std_logic;
10      RDA: in std_logic;
11      data_in: in std_logic_vector(7 downto 0);
12      data_out: out std_logic_vector(7 downto 0);
13      RD: out std_logic
14  );
15 end B;
16
17 architecture Behavioral of B is
18 component Control_Unit_B is
19     Port (
20         clock: in std_logic;
21         reset: in std_logic;
22         count_out : in std_logic_vector(2 downto 0);
23         RDA: in std_logic;
24         RD: out std_logic;
25         count_in :out std_logic;
26         address: out std_logic_vector(2 downto 0);
27         load: out std_logic;
28         write_mem: out std_logic
29     );
30
31 end component;
32 --Contatore
33 component cont_mod8 is
34     generic(
35         Log: integer := 3
36     );
```

```
37
38     port( clock, reset: in std_logic;
39           count_in: in std_logic;
40           count: out std_logic_vector(Log-1 downto 0));
41 end component;
42 --MEMORIA
43 component mem is
44 generic(
45     lunghezza_indirizzo: integer := 3;
46     lunghezza_dati: integer := 8;
47     number_of_location: integer:= 8
48 );
49 port(
50     clock: in std_logic;
51     reset: in std_logic;
52     read: in std_logic;
53     write: in std_logic;
54     address: in std_logic_vector((lunghezza_indirizzo - 1)
55                                   downto 0);
56     data_in: in std_logic_vector((lunghezza_dati- 1) downto 0)
57         ;
58     data_output: out std_logic_vector ((lunghezza_dati - 1)
59                                         downto 0)
60 );
61 end component;
```

```
62
63
64
65 signal count_out, address: std_logic_vector(2 downto 0) := (others
    => '0');
66 signal count_in, load_M, write_M: std_logic := '0';
67
68 begin
69 CU_B: Control_Unit_B port map(clock, reset, count_out, RDA, RD,
    count_in, address, load_M, write_M);
70 Contatore_B: cont_mod8 generic map(3) port map(clock, reset,
    count_in, count_out);
71 MEMORIA_B: mem generic map(3, 8, 8) port map(clock, reset, load_M,
    write_M, address, data_in, data_out);
72
73 end Behavioral;
```

Implementazione Top Module:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6 entity Top_module is
7     Port (
8
9     CLOCK: in std_logic;
10    RESET: in std_logic;
11    WRITE_BUTTON : in std_logic;
```

```
12  FE: out std_logic;
13  OE:out std_logic;
14  PE:out std_logic;
15  USCITA : out std_logic_vector(7 downto 0)
16
17  );
18  end Top_module;
19
20  architecture Behavioral of Top_module is
21  component A is
22      Port (
23          CLOCK: in std_logic;--master clock
24          RESET: in std_logic;--reset
25          WRITE: in std_logic;-- segnale per l avvio dell
                operazione di lettura da rom
26          TBE: in std_logic; --uscita derlla uart in trasmissione
27          WR: out std_logic;--segnale da connettere alla uart
28          data_out: out std_logic_vector(7 downto 0)
29
30      );
31  end component;
32  --NODO B
33  component B is
34      Port (
35          clock: in std_logic;
36          reset: in std_logic;
37          RDA: in std_logic;
38          data_in: in std_logic_vector(7 downto 0);
```

```
39     data_out:out  std_logic_vector(7 downto 0);
40     RD: out std_logic
41 );
42 end component;
43
44 component Rs232RefComp is
45     Port (
46         TXD  : out std_logic;
47         RXD  : in  std_logic;
48         CLK  : in  std_logic;
49         DBIN : in  std_logic_vector(7 downto 0);
50         DBOUT: out std_logic_vector(7 downto 0);
51         RDA  : inout std_logic;
52         TBE  : inout std_logic;
53         RD   : in  std_logic;
54         WR   : in  std_logic;
55         PE   : out std_logic;
56         FE   : out std_logic;
57         OE   : out std_logic;
58         RST  : in  std_logic := '0'
59     );
60 end component;
61
62 component clock_divider is
63     generic(
64         clock_frequency_in : integer := 100000000;--
65         100MHz
66         clock_frequency_out : integer := 500 --500 Hz
```

```

66         );
67     Port ( clock_in : in  STD_LOGIC;
68           reset  : in  STD_LOGIC;
69           clock_out : out STD_LOGIC);
70 end component;
71 signal txd_rxd: std_logic;
72 signal tbe: std_logic;
73 signal WR : std_logic:='0';
74 signal dato_trasmesso: std_logic_vector(7 downto 0):=(others
    =>'0');
75 signal clock_diviso: std_logic:='0';
76 signal RDA:std_logic:='0';
77 signal RD: std_logic;
78 signal DATA_OUT,OUT_B: std_logic_vector (7 downto 0);
79
80 begin
81
82 NODO_A : A port map(CLOCK,RESET,WRITE_BUTTON,tbe,WR,
    dato_trasmesso);
83
84 DIVISORE_CLOCK:clock_divider generic map( 100000000,27e6) port
    map(CLOCK,RESET,clock_diviso);
85 --UART in trasmissione
86 UART_T: Rs232RefComp port map (
87     TXD => txd_rxd,
88     RXD => '0',
89     CLK => clock_diviso,
90     DBIN => dato_trasmesso,

```

```
91         TBE => tbe,
92         RD  => '1',
93         WR  => WR,
94         RST => reset
95     );
96 --UART in ricezione
97 UART_R:Rs232RefComp port map (
98     RXD => txd_rxd,
99     CLK => clock_diviso,
100     DBIN => dato_trasmesso,
101     DBOUT=> DATA_OUT,
102     RD  => RD,
103     WR  => '0',
104     RST => reset,
105     FE=>FE,
106     OE=>OE,
107     PE=>PE,
108     RDA=>RDA
109 );
110
111 --NODO B
112 NODO_B: B port map (CLOCK, RESET, RDA, DATA_OUT, OUT_B, RD) ;
113 USCITA<=DATA_OUT;
114
115 end Behavioral;
```

Simulazione

Simulazione Top Module:

```
1      library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6  entity TB is
7  -- Port ( );
8  end TB;
9
10 architecture Behavioral of TB is
11 component Top_module is
12     Port (
13
14     CLOCK: in std_logic;
15     RESET: in std_logic;
16     WRITE_BUTTON : in std_logic;
17     FE: out std_logic;
18     OE:out std_logic;
19     PE:out std_logic;
20     USCITA : out std_logic_vector(7 downto 0)
21
22     );
23 end component;
24
25
26
```

```
27 signal uscita: std_logic_vector(7 downto 0) := (others => '0');
28 signal CLK, RST, wrt, FE, OE, PE: std_logic := '0';
29 constant clock_period: time := 10 ns;
30
31
32 begin
33   CLK_process: process
34   begin
35     CLK <= '0';
36     wait for clock_period/2;
37     CLK <= '1';
38     wait for clock_period/2;
39   end process;
40
41   UUT: Top_module port map (CLK, RST, wrt, FE, OE, PE, uscita);
42
43   STIM_PROCESS: process
44
45   begin
46
47     RST <= '1';
48
49     wait for 10 ns;
50
51     RST <= '0';
52     wrt <= '1';
53
54     wait for 10 ns;
```

```

55 wrt<='0';
56 wait for 70*clock_period;
57
58 wrt<='1';
59 wait for 10 ns;
60 wrt<='0';
61 wait ;
62 end process;
63
64
65
66
67 end Behavioral;

```

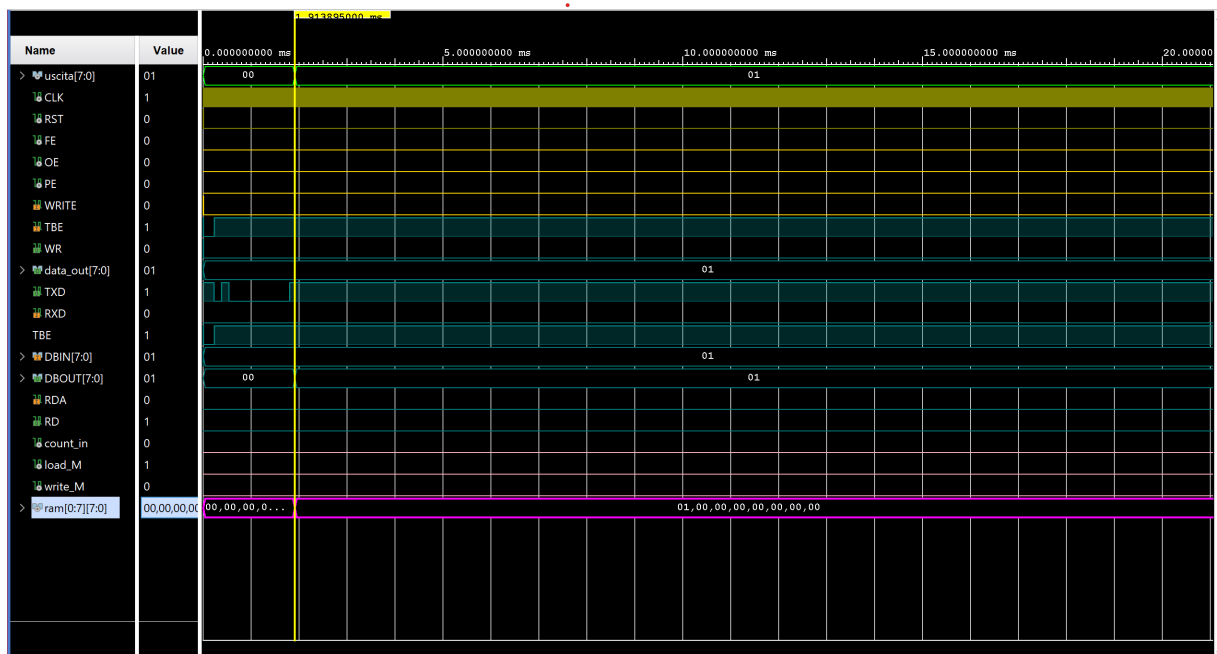


Figura 6.4: Simulazione 10

6.1.2 Esercizio 10bis

Dopo aver simulato il comportamento del sistema, lo si implementi su board usando un bottone per il segnale di WR in A, un bottone per il segnale di RD in B, e i display per la visualizzazione del dato correntemente trasmesso e memorizzato in MEM. Si testi l'errore di overrun.

Progetto e architettura

Partendo dall'implementazione dell'esercizio precedente, sono stati aggiunti due bottoni, uno per il segnale di WR e uno per il segnale di RD ed è stato aggiornato opportunamente il constraint file. Per testare gli errori dovuti alla trasmissione abbiamo aggiunto tre led (uno per l'overrun, uno per il parity error e uno per il frame error). In particolar modo il nostro scopo era testare l'errore di overrun, per fare ciò è bastato far sì che la Uart in trasmissione fosse più veloce di quella in ricezione.

Implementazione

Implementazione Top Module con clock divider per A per far sì che non si verifichi l'errore di overrun:

```
1      library IEEE;  
2 use IEEE.STD_LOGIC_1164.ALL;  
3  
4  
5  
6 entity On_chip_ARCH is
```

```
7 Port (
8     CLOCK: IN std_logic;
9     RESET: IN std_logic;
10    WRITE_BUTTON: IN STD_LOGIC;
11    READ_BUTTON: IN STD_LOGIC;
12    cathodes_out: out std_logic_vector(7 downto 0);
13    anodes_out: out std_logic_vector (7 downto 0);
14    led: out std_logic_vector(2 downto 0)
15
16 );
17 end On_chip_ARCH;
18
19 architecture Behavioral of On_chip_ARCH is
20 --dichiarazione segnali
21 signal dato_consumato: std_logic_vector(7 downto 0):=(others
    =>'0');
22 signal cleared_wrt,cleared_read: std_logic:='0';
23 signal value_in:std_logic_vector(31 downto 0):=(others=>'0');
24
25 component Top_module is
26     Port (
27
28     CLOCK: in std_logic;
29     RESET: in std_logic;
30     WRITE_BUTTON : in std_logic;
31     --Aggiungere bottone per abilitare la lettura
32     READ_SW: in std_logic;
33     FE: out std_logic;
```

```

34 OE:out std_logic;
35 PE:out std_logic;
36 USCITA : out std_logic_vector(7 downto 0)
37
38 );
39 end component;
40
41 --Button Debouncer
42 component ButtonDebouncer is
43     generic (
44         CLK_period:    integer := 10;  -- periodo del clock
45         della board 10 nanosecondi
46         btn_noise_time: integer := 6500000 --intervallo di
47         tempo in cui si ha l'oscillazione del bottone
48         --assumo che duri 6.5
49         ms=6500microsec
50         =6500000ns
51     );
52     Port ( RST : in STD_LOGIC;
53           CLK  : in STD_LOGIC;
54           BTN  : in STD_LOGIC;
55           CLEARED_BTN : out STD_LOGIC);
56 end component;
57
58 --Display a 7 segmenti
59 component display_seven_segments is
60     Generic(
61         CLKIN_freq : integer := 100000000;

```

```

58         CLKOUT_freq : integer := 500
59     );
60     Port ( CLK : in  STD_LOGIC;
61           RST  : in  STD_LOGIC;
62           VALUE : in  STD_LOGIC_VECTOR (31 downto 0);
63           ENABLE : in  STD_LOGIC_VECTOR (7 downto 0); --
64               decide quali cifre abilitare
65           DOTS : in  STD_LOGIC_VECTOR (7 downto 0); -- decide
66               quali punti visualizzare
67           ANODES : out STD_LOGIC_VECTOR (7 downto 0);
68           CATHODES : out STD_LOGIC_VECTOR (7 downto 0));
69
70 end component;
71
72 begin
73
74     --DEBOUNCER_wrt: ButtonDebounce generic map(10,6500000) port
75         map (RESET,CLOCK,WRITE_BUTTON,cleared_wrt);
76
77     --DEBOUNCER_read: ButtonDebounce generic map(10,6500000) port
78         map (RESET,CLOCK,READ_BUTTON,cleared_read);
79
80     ARCHITETTURA TRASFER: Top_module port map(CLOCK,RESET,
81         WRITE_BUTTON,READ_BUTTON,led(2),led(1),led(0),dato_consumato
82     );
83
84     value_in<= "00000000000000000000000000" & dato_consumato;
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```
80 --Display a 7 segmenti
81
82 Display_a_sette_segmenti: display_seven_segments generic map
    (1000000000,500)
83 port map(CLK=>CLOCK,RST=>RESET,VALUE=>value_in,ENABLE=>"
    11111111",DOTS=>"00000000",ANODES=>anodes_out,CATHODES=>
    cathodes_out);
84
85
86 end Behavioral;
```

Per testare l'errore di overrun è necessario modificare la tempificazione del nodo A.

Simulazione

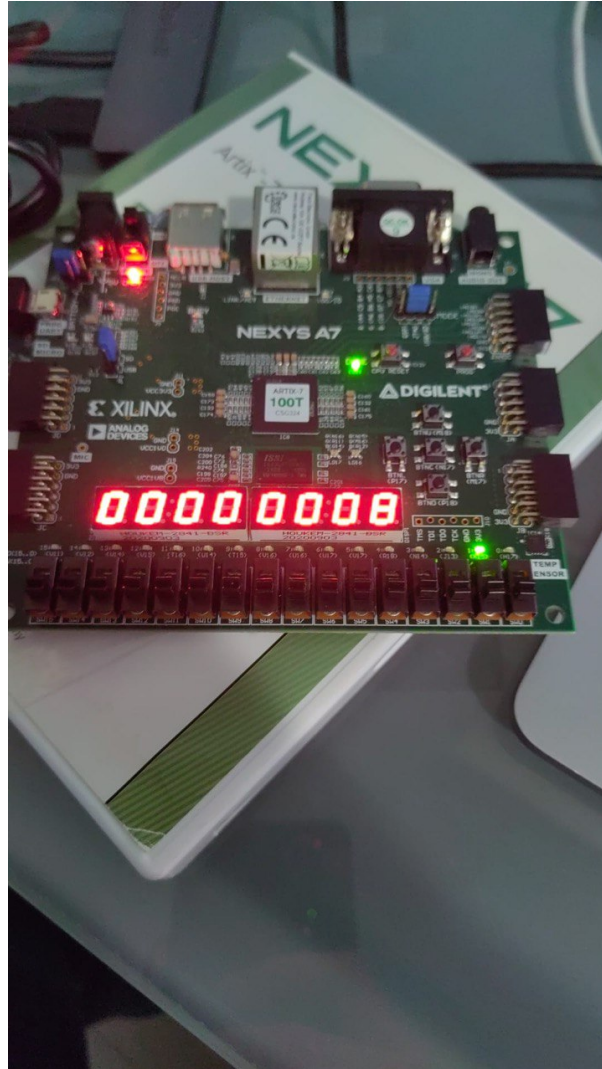


Figura 6.5:

Capitolo 7

SWITCH MULTISTADIO

7.1 Esercizio 11

7.1.1 Esercizio 11.1

Progettare ed implementare in VHDL uno switch multistadio secondo il modello omega network. Lo switch deve consentire lo scambio di messaggi di 2 bit ciascuno da un nodo sorgente a un nodo destinazione in una rete con 4 nodi, implementando uno schema a priorità fissa fra i nodi (es. nodo 1 più prioritario, con priorità decrescenti fino al nodo 4).

Progetto e architettura

Il nostro scopo è implementare uno switch multistadio seguendo il modello omega network. Questa topologia consiste in una rete di interconnessione $N \times N$, con $\log_2 N$ stadi, basata sull'algoritmo perfect shuffling. Tale algoritmo si ispira al gioco delle carte, divide il "mazzo" in metà uguali e combina il

nodo i -esimo della prima metà con il nodo i -esimo della seconda. Dopo $\log_2 N$ stadi i nodi si ritroveranno nell'ordine di partenza.

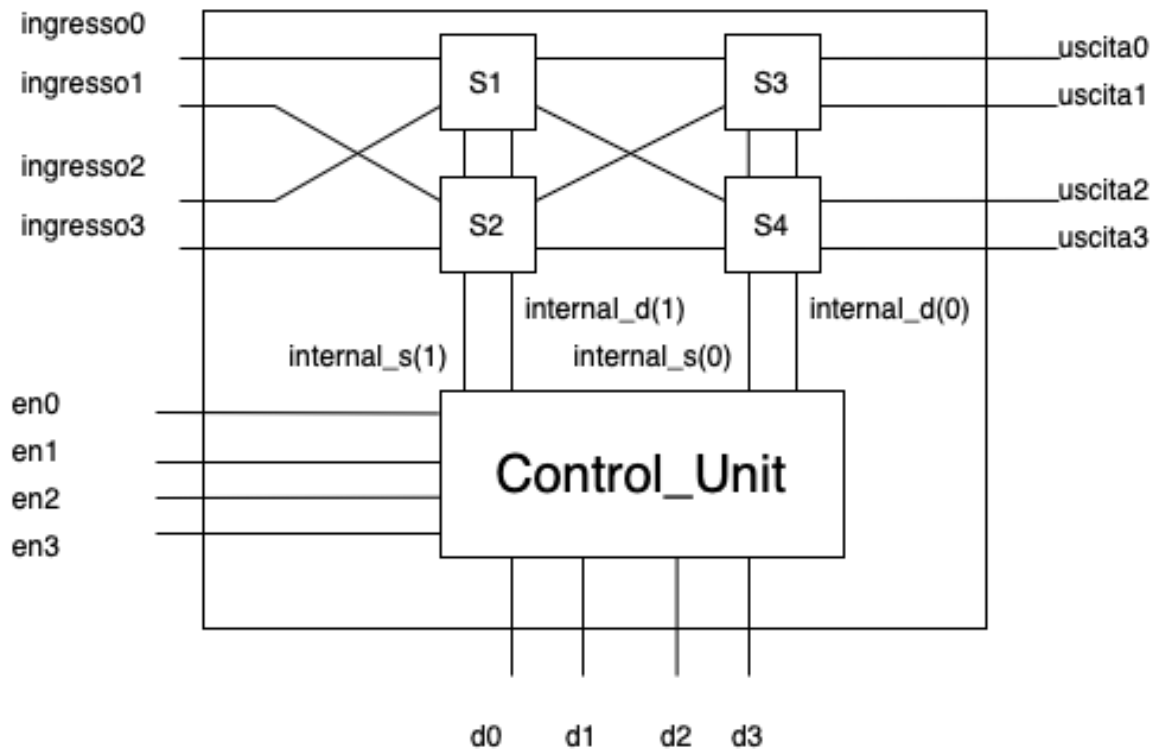


Figura 7.1: Omega Network

Il costrutto base di questa architettura è uno switch. Nel nostro caso scegliamo di utilizzare uno switch elementare composto da un multiplexer 2:1 e un demultiplexer 1:2. Questa scelta risulta più scalabile.

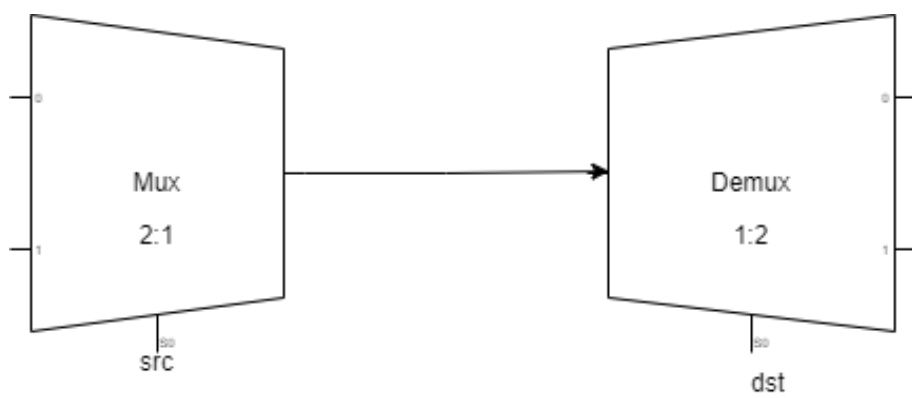


Figura 7.2: Switch

Implementazione

Implementazione switch:

```
1
2 library IEEE;
3 use IEEE.STD_LOGIC_1164.ALL;
4
5
6 entity switch is
7     Port (
8         x1: in std_logic_vector(1 downto 0) ;
9         x2: in std_logic_vector(1 downto 0) ;
10        src: in std_logic ;
11        dst: in std_logic ;
12        y1:  out std_logic_vector(1 downto 0);
13        y2:  out std_logic_vector (1 downto 0)
14    );
15 end switch;
16
17 architecture structural of switch is
18
19 component mux_2_1 is
20
21     port(    a0   : in std_logic_vector(1 downto 0);
22            a1   : in std_logic_vector(1 downto 0);
23            s    : in STD_LOGIC;
24            y    : out std_logic_vector(1 downto 0)
25    );
26 end component ;
```

```
27
28 component demux_1_2 is
29     Port ( ingresso : in std_logic_vector(1 downto 0);
30           selettore  : in std_logic;
31           output1    : out std_logic_vector(1 downto 0);
32           output2    : out std_logic_vector(1 downto 0)
33           );
34 end component ;
35
36 signal uscita_mux: std_logic_vector(1 downto 0) := (others
37     =>'0');
38
39 begin
40
41 MUX: mux_2_1 port map(
42     a0=>x1,
43     a1=>x2,
44     s=>src,
45     y=>uscita_mux
46 );
47
48 DEMUX: demux_1_2 port map(
49     ingresso => uscita_mux,
50     selettore => dst,
51     output1  => y1,
52     output2  => y2
53 );
```

```
54 end structural;
```

Implementazione Control Unit:

```

1      library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity control_unit is
5      Port (
6          en0:in std_logic ;
7          en1:in std_logic ;
8          en2:in std_logic ;
9          en3:in std_logic ;
10         input0: in std_logic_vector(1 downto 0);
11         input1: in std_logic_vector(1 downto 0);
12         input2: in std_logic_vector(1 downto 0);
13         input3: in std_logic_vector(1 downto 0);
14         d0,d1,d2,d3: in std_logic_vector (1 downto 0);
15         s: out std_logic_vector(1 downto 0);
16         d:out std_logic_vector (1 downto 0)
17     );
18 end control_unit;
19
20 architecture dataflow of control_unit is
21
22     signal p0,p1,p2,p3,p4: std_logic:='0';
23     signal int_src: std_logic_vector(1 downto 0):=(others=>'0');
24 begin
25
26     p0<=en0;
```

```
27 p1<=(not en0) and en1;
28 p2<=(not en0) and (not en1) and en2;
29 p3<=(not en0) and (not en1) and (not en2) and en3;
30
31
32 int_src(1) <= ((not p0) and (not p1) and (not p2) and p3) or
    ((not p0) and (not p1) and p2 and (not p3));
33 int_src(0) <= ((not p0) and (not p1) and (not p2) and p3) or
    ((not p0) and p1 and (not p2) and (not p3));
34
35 s<=int_src;
36
37 --gestione destinazione
38 with int_src select
39     d<=
40     d0 when "00",
41     d1 when "01",
42     d2 when "10",
43     d3 when "11",
44     "UU" when others;
45
46
47 end dataflow;
```

Implementazione omega network:

```
1     library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity omega_network is
```

```
5  Port (
6  ingresso0: in std_logic_vector(1 downto 0);
7  ingresso1: in std_logic_vector(1 downto 0);
8  ingresso2: in std_logic_vector(1 downto 0);
9  ingresso3: in std_logic_vector(1 downto 0);
10 enable0:in std_logic ;
11 enable1:in std_logic ;
12 enable2:in std_logic ;
13 enable3:in std_logic ;
14 d0,d1,d2,d3: in std_logic_vector (1 downto 0);
15 uscita0: out std_logic_vector(1 downto 0);
16 uscita1: out std_logic_vector(1 downto 0);
17 uscita2: out std_logic_vector(1 downto 0);
18 uscita3: out std_logic_vector(1 downto 0)
19 );
20 end omega_network;
21
22 architecture Behavioral of omega_network is
23
24
25
26 component switch is
27   Port (
28     x1: in std_logic_vector(1 downto 0) ;
29     x2: in std_logic_vector(1 downto 0) ;
30     src: in std_logic ;
31     dst: in std_logic ;
32     y1: out std_logic_vector(1 downto 0);
```

```
33   y2: out std_logic_vector (1 downto 0)
34   );
35 end component;
36
37 component control_unit is
38   Port (
39     en0:in std_logic ;
40     en1:in std_logic ;
41     en2:in std_logic ;
42     en3:in std_logic ;
43     input0: in std_logic_vector(1 downto 0);
44     input1: in std_logic_vector(1 downto 0);
45     input2: in std_logic_vector(1 downto 0);
46     input3: in std_logic_vector(1 downto 0);
47     d0,d1,d2,d3: in std_logic_vector (1 downto 0);
48     s: out std_logic_vector(1 downto 0);
49     d:out std_logic_vector (1 downto 0)
50   );
51 end component;
52
53 signal internal_s,internal_d,output0,output1, output2,output3:
54     std_logic_vector(1 downto 0) := (others=>'0');
55
56
57 begin
58
59 CU: control_unit port map(enable0,enable1,enable2, enable3,
60     ingresso0,ingresso1,ingresso2,ingresso3,d0,d1,d2,d3,
61     internal_s,internal_d);
```

```
58
59 --primo stadio
60 S1: switch port map(
61     x1=> ingresso0,
62     x2=>ingresso2,
63     src=>internal_s(1),
64     dst=>internal_d(1),
65     y1=>output0,
66     y2=>output2
67
68 );
69 S2: switch port map(
70     x1=> ingresso1,
71     x2=>ingresso3,
72     src=>internal_s(1),
73     dst=>internal_d(1),
74     y1=>output1,
75     y2=>output3
76
77 );
78
79 --secondo stadio
80 S3: switch port map(
81     x1=> output0,
82     x2=>output1,
83     src=>internal_s(0),
84     dst=>internal_d(0),
85     y1=>uscita0,
```

```
86     y2=>uscital
87
88 );
89 S4: switch port map(
90     x1=> output2,
91     x2=> output3,
92     src=>internal_s(0),
93     dst=>internal_d(0),
94     y1=>uscita2,
95     y2=>uscita3
96
97 );
98
99 end Behavioral;
```

Simulazione

Di seguito è riportata la simulazione dell'omega network:

```
1     library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity omega_network_tb is
5 -- Port ( );
6 end omega_network_tb;
7
8 architecture Behavioral of omega_network_tb is
9
10
```

```

11
12 component omega_network is
13     Port (
14         ingresso0: in std_logic_vector(1 downto 0);
15         ingresso1: in std_logic_vector(1 downto 0);
16         ingresso2: in std_logic_vector(1 downto 0);
17         ingresso3: in std_logic_vector(1 downto 0);
18         enable0:in std_logic ;
19         enable1:in std_logic ;
20         enable2:in std_logic ;
21         enable3:in std_logic ;
22         d0,d1,d2,d3: in std_logic_vector (1 downto 0);
23         uscita0: out std_logic_vector(1 downto 0);
24         uscita1: out std_logic_vector(1 downto 0);
25         uscita2: out std_logic_vector(1 downto 0);
26         uscita3: out std_logic_vector(1 downto 0)
27     );
28 end component;
29
30
31 signal en_0,en_1,en_2,en_3: std_logic:='0';
32 signal d0,d1,d2,d3,ingresso0,ingresso1,ingresso2,ingresso3,u0,
    u1,u2,u3: std_logic_vector(1 downto 0):=(others=>'0');
33
34 begin
35
36 Omega_n: omega_network port map( ingresso0,ingresso1,ingresso2
    ,ingresso3,en_0,en_1,en_2,en_3, d0,d1,d2,d3,u0,u1,u2,u3);

```

```
37
38 stim: process
39     begin
40         en_0<='0';
41         en_1<='0';
42         en_2<='0';
43         en_3<='1';
44         ingresso0<="00";
45         ingresso1<="01";
46         ingresso2<="10";
47         ingresso3<="11";
48         d0<="11";
49         d1<="10";
50         d2<="01";
51         d3<="00";
52     wait;
53     end process;
54 end Behavioral;
```

7.1.2 Esercizio 11.2

Rimuovendo l'ipotesi di lavorare secondo uno schema a priorità fissa fra i nodi e considerando una rete di 8 nodi, lo switch deve gestire eventuali conflitti generati da collisioni con un meccanismo a scelta dello studente.

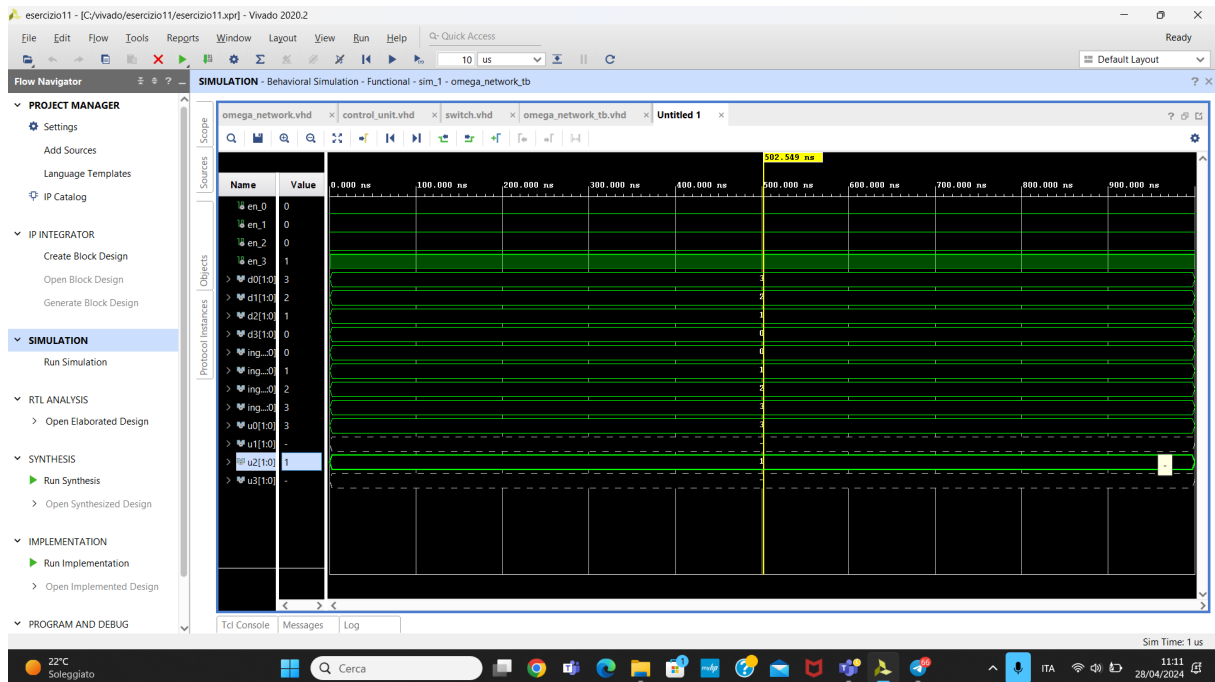


Figura 7.3: Simulazione11.1

Progetto e architettura

L'approccio scelto per gestire eventuali conflitti è quello dello **Store-and-forward**. Il metodo dello store and forward prevede che l'intero messaggio (in questo caso 2 bit) sia ricevuto e archiviato da uno switch prima di essere inoltrato al nodo successivo, quando questo è disponibile.

Per poter implementare ciò abbiamo modificato la struttura dello switch del precedente punto. Quest'ultimo non sarà più composto da mux e demux collegati tra loro ma sono stati aggiunti 4 shift register, una Control Unit, e un contatore che funge da scheduler.

In seguito è riportata l'architettura dello switch che mostra come questi elementi sono connessi tra di loro:

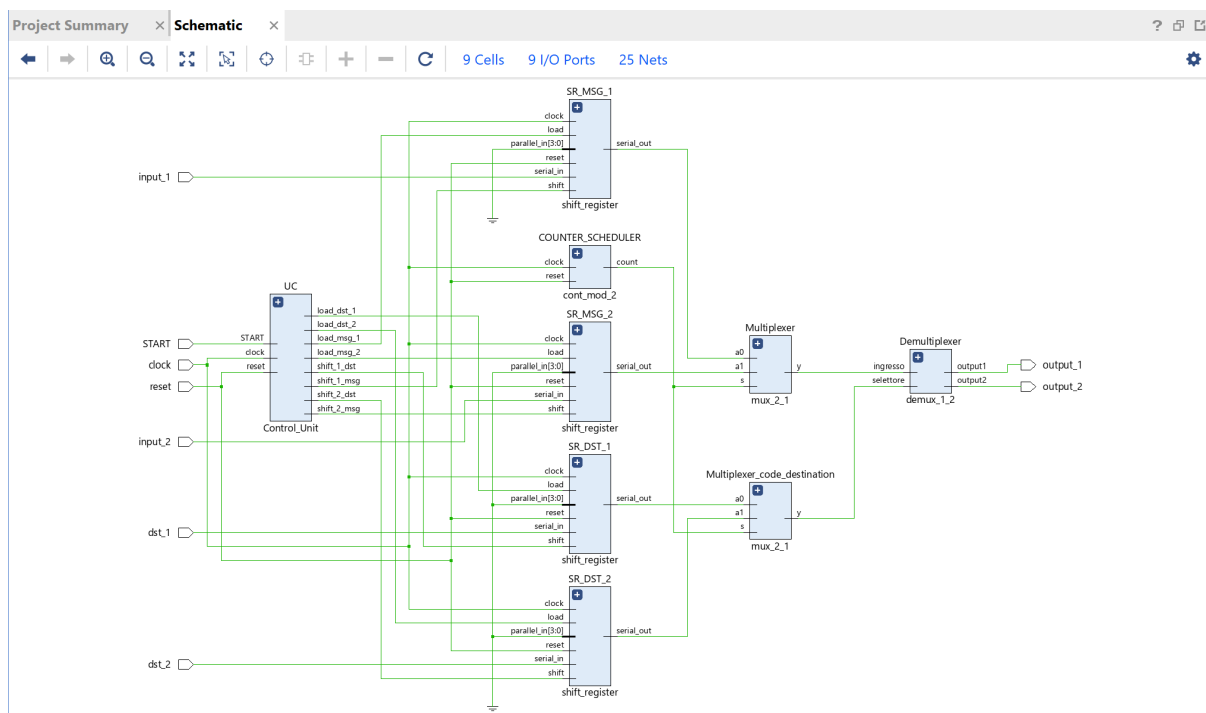


Figura 7.4: Switch

Questo componente è pensato per ricevere in ingresso il messaggio e la destinazione. Avendo rimosso l'ipotesi di una comunicazione a priorità fissa, ciascun messaggio verrà inserito all'interno di una "coda" (lo shift register) che conserverà il messaggio per evitarne la perdita nel caso in cui il nodo destinazione sia momentaneamente occupato.

Inoltre, per garantire "fairness" tra i vari nodi sorgente è stato sfruttato il contatore per gestire la schedulazione tra i messaggi in coda.

Il modello dell'architettura generale è riportato in seguito:

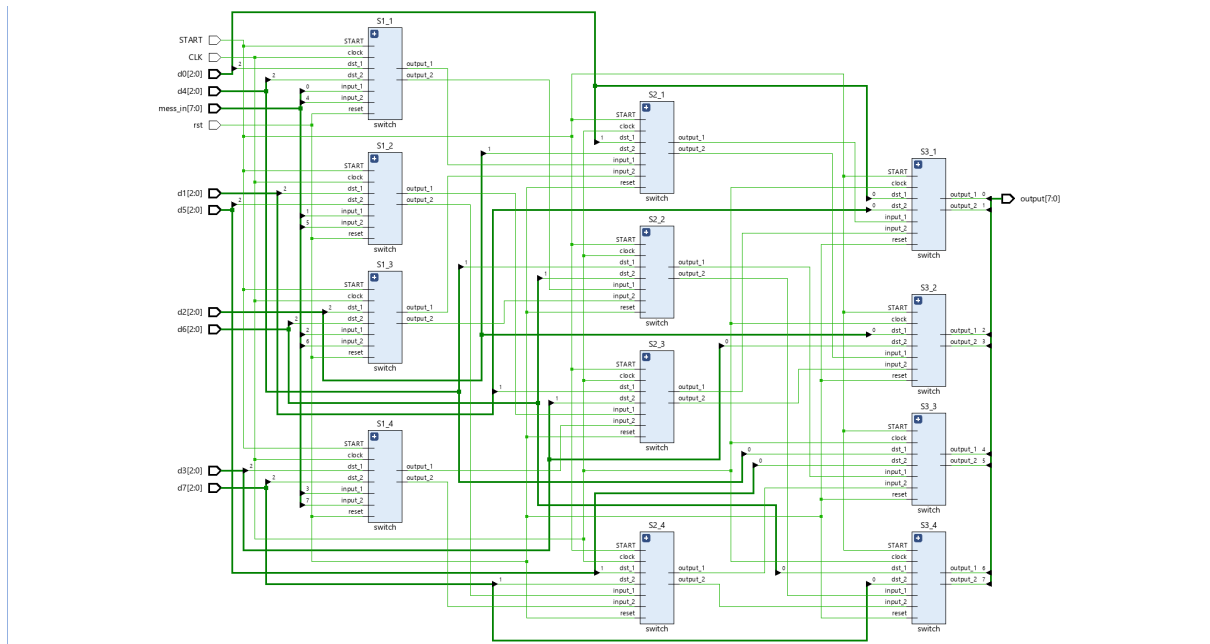


Figura 7.5: Top Module Omega Network con 8 nodi

Implementazione

Implementazione Switch:

```

1      library IEEE;
2      use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6      entity switch is
7      Port (
8          START, clock, reset: in std_logic;
9          input_1, input_2: in std_logic;
10         dst_1, dst_2: in std_logic;
11         output_1, output_2: out std_logic
12     );

```

```
13 end switch;
14
15 architecture Structural of switch is
16 --CONTATORE
17 component cont_mod_2 is
18
19     port (
20         clock, reset: in std_logic;
21         count: out std_logic);
22 end component;
23
24 --MUX_2_1
25 component mux_2_1 is
26
27     port (    a0   : in std_logic;
28             a1   : in std_logic;
29             s    : in STD_LOGIC;
30             y    : out std_logic
31         );
32 end component;
33
34 --DEMUX_1_2
35 component demux_1_2 is
36     Port ( ingresso : in std_logic;
37           selettore : in std_logic;
38           output1  : out std_logic;
39           output2  : out std_logic
40         );
```

```
41 end component;
42
43 --CONTROL_UNIT
44 component Control_Unit is
45   Port (
46     START: in std_logic;
47     clock, reset: in std_logic;
48     shift_1_msg: out std_logic;
49     shift_2_msg: out std_logic;
50     shift_1_dst, shift_2_dst: out std_logic;
51     load_msg_1, load_msg_2: out std_logic;
52     load_dst_1, load_dst_2: out std_logic
53   );
54 end component;
55
56
57 --SR
58 component shift_register is
59   port (
60     parallel_in: in std_logic_vector(3 downto 0);
61     serial_in: in std_logic;
62     clock, reset, load, shift: in std_logic;
63     serial_out: out std_logic
64   );
65 end component;
66
67
68 signal shift_msg_1, shift_msg_2, shift_dst_1, shift_dst_2,
```

```
load_msg_1,load_msg_2,load_dst_1,load_dst_2,count: std_logic
:= '0';

69 signal init_shift: std_logic_vector(3 downto 0) := (others=>'0')
;

70 signal out_mux,out_mux_2,out_shift_1,out_shift_2: std_logic
:= '0';

71 signal out_shift_dst_1,out_shift_dst_2: std_logic:= '0';

72

73 begin

74 init_shift<="0000";

75

76 UC:Control_Unit port map(START,clock,reset,shift_msg_1,
    shift_msg_2,shift_dst_1,shift_dst_2,load_msg_1,load_msg_2,
    load_dst_1,load_dst_2);

77

78 COUNTER_SCHEDULER:cont_mod_2 port map(clock,reset,count);

79

80 SR_MSG_1: shift_register port map(init_shift,input_1,clock,
    reset,load_msg_1,shift_msg_1,out_shift_1);

81 SR_MSG_2: shift_register port map(init_shift,input_2,clock,
    reset,load_msg_2,shift_msg_2,out_shift_2);

82

83 SR_DST_1:shift_register port map(init_shift,dst_1,clock,reset,
    load_dst_1,shift_dst_1,out_shift_dst_1);

84 SR_DST_2:shift_register port map(init_shift,dst_2,clock,reset,
    load_dst_2,shift_dst_2,out_shift_dst_2);

85 Multiplexer:mux_2_1 port map(out_shift_1,out_shift_2,count,
    out_mux);
```

```
86
87 Multiplexer_code_destination:mux_2_1 port map(out_shift_dst_1,
      out_shift_dst_2,count,out_mux_2);
88
89 Demultiplexer: demux_1_2 port map(out_mux,out_mux_2,output_1,
      output_2);
90
91
92
93 end Structural;
```

Implementazione Omega Network:

```
1      library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6 entity omega_net_8 is
7 Port (
8     START,CLK,rst: in std_logic;
9     mess_in: in std_logic_vector(7 downto 0);
10    d0,d1,d2,d3,d4,d5,d6,d7: in std_logic_vector(2 downto 0);
11    output: out std_logic_vector(7 downto 0)
12
13 );
14 end omega_net_8;
15
16 architecture Behavioral of omega_net_8 is
17
```

```

18 component switch is
19   Port (
20       START,clock,reset: in std_logic;
21       input_1,input_2: in std_logic;
22       dst_1,dst_2: in std_logic;
23       output_1,output_2: out std_logic
24   );
25 end component;
26 --Segnali di appoggio uscite del primo livello
27 signal u0,u1,u2,u3,u4,u5,u6,u7: std_logic:='0';
28 --Segnali di appoggio uscite del secondo livello
29 signal o0,o1,o2,o3,o4,o5,o6,o7: std_logic:='0';
30
31 begin
32 --PRIMO LIVELLO
33 S1_1:switch port map (START,CLK,rst,mess_in(0),mess_in(4),d0(2)
34     ,d4(2),u0,u1);
35 S1_2: switch port map (START,CLK,rst,mess_in(1),mess_in(5),d1
36     (2),d5(2),u2,u3);
37 S1_3: switch port map (START,CLK,rst,mess_in(2),mess_in(6),d2
38     (2),d6(2),u4,u5);
39 S1_4:switch port map (START,CLK,rst,mess_in(3),mess_in(7),d3(2)
40     ,d7(2),u6,u7);
41
42 --SECONDO LIVELLO
43 S2_1:switch port map (START,CLK,rst,u0,u4,d0(1),d2(1),o0,o1);
44 S2_2:switch port map (START,CLK,rst,u1,u5,d4(1),d6(1),o2,o3);

```

```
42 S2_3:switch port map (START,CLK,rst,u2,u6,d1(1),d3(1),o4,o5);
43 S2_4:switch port map (START,CLK,rst,u3,u7,d5(1),d7(1),o6,o7);
44
45 --Output Layer
46 S3_1:switch port map (START,CLK,rst,o0,o4,d0(0),d1(0),output(0)
    ,output(1));
47 S3_2:switch port map (START,CLK,rst,o1,o5,d2(0),d3(0),output(2)
    ,output(3));
48 S3_3:switch port map (START,CLK,rst,o2,o6,d4(0),d5(0),output(4)
    ,output(5));
49 S3_4:switch port map (START,CLK,rst,o3,o7,d6(0),d7(0),output(6)
    ,output(7));
50
51
52
53 end Behavioral;
```

Simulazione

Di seguito è riportata la simulazione dell'omega network:

```
1      library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4
5  entity Omega_TB is
6  -- Port ( );
7  end Omega_TB;
8
```

```
9 architecture Behavioral of Omega_TB is
10 component omega_net_8 is
11 Port (
12     START,CLK,rst: in std_logic;
13     mess_in: in std_logic_vector(7 downto 0);
14     d0,d1,d2,d3,d4,d5,d6,d7: in std_logic_vector(2 downto 0);
15     output: out std_logic_vector(7 downto 0)
16
17 );
18 end component;
19
20 signal START,clock,rst:std_logic:='0';
21 signal messaggio_inviato,messaggio_ricevuto: std_logic_vector
    (7 downto 0):=(others=>'0');
22 signal d0,d1,d2,d3,d4,d5,d6,d7: std_logic_vector(2 downto 0)
    :=(others=>'0');
23 constant clock_period: time:=10 ns;
24
25 begin
26 CLK_process: process
27 begin
28     clock<='0';
29     wait for clock_period/2;
30     clock<='1';
31     wait for clock_period/2;
32 end process;
33
34
```

```
35
36 UUT:omega_net_8 port map (START,clock,rst,messaggio_inviato,d0,
    d1,d2,d3,d4,d5,d6,d7,messaggio_ricevuto);
37
38 STIM_proc: process
39 begin
40 wait for 10 ns;
41 rst<='1';
42 wait for 10 ns;
43 rst<='0';
44 START<='1';
45
46 messaggio_inviato<="10000000";
47 d7<="000";
48 d0<="001";
49 d1<="000";
50 d2<="100";
51 d4<="100";
52 d5<="100";
53 d6<="100";
54 d3<="100";
55
56
57 wait for 300 ns;
58
59 wait;
60 end process;
61 end Behavioral;
```

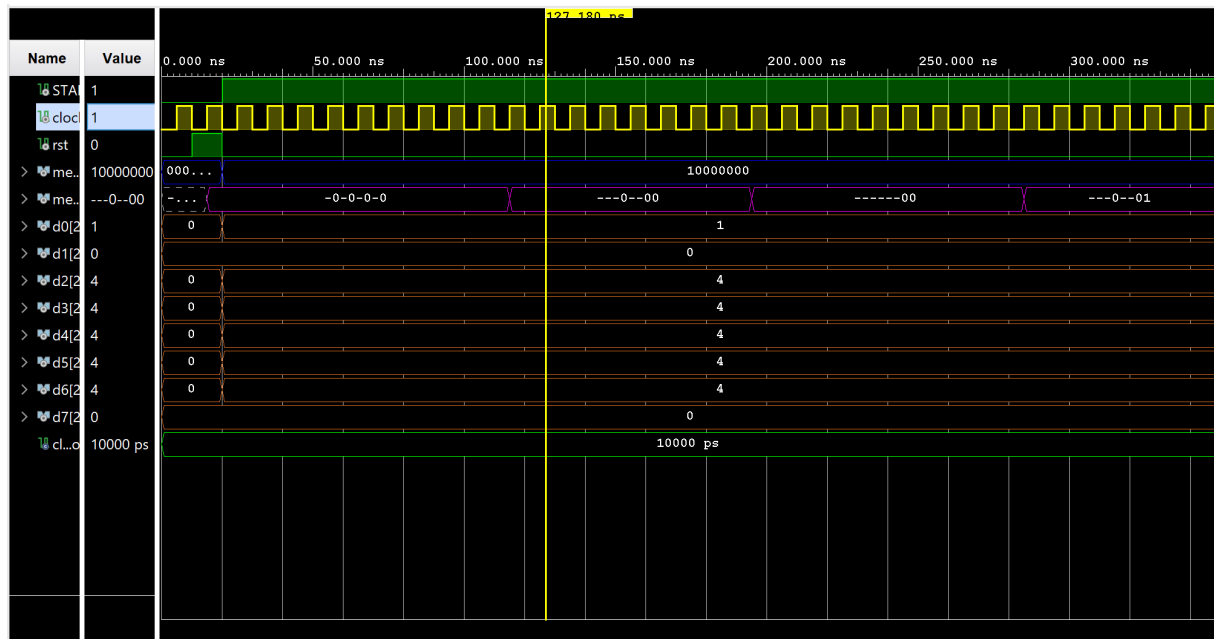


Figura 7.6: Simulazione Omega Network

7.1.3 Esercizio 11.3

Si implementi un protocollo di handshaking semplice regolato da una coppia di segnali (pronto a inviare/pronto a ricevere) per l'invio di ciascun messaggio fra due nodi.

Progetto e architettura

Per facilitare l'implementazione del protocollo di handshaking siamo tornati all'architettura dell'Omega Network dell'esercizio 11.1.

Partendo dall'Omega Network stessa abbiamo collegato dei registri ai vari ingressi e alle varie uscite dei nodi, rispettivamente per gli ingressi dei registri lettura scrittura (txd) e per le uscite dei registri di sola scrittura (rxid).

Per gestire la comunicazione abbiamo implementato un nodo trasmettitore e un nodo ricevitore con le rispettive "Control Unit". Quest'ultime vanno a gestire il protocollo di handshaking tramite i segnali **req** e **ack** (pronto a inviare/pronto a ricevere).

Infine, la "Control Unit" gestisce la logica presente alla base dell'Omega Network, come nei punti precedenti.

In seguito, è riportata l'architettura:

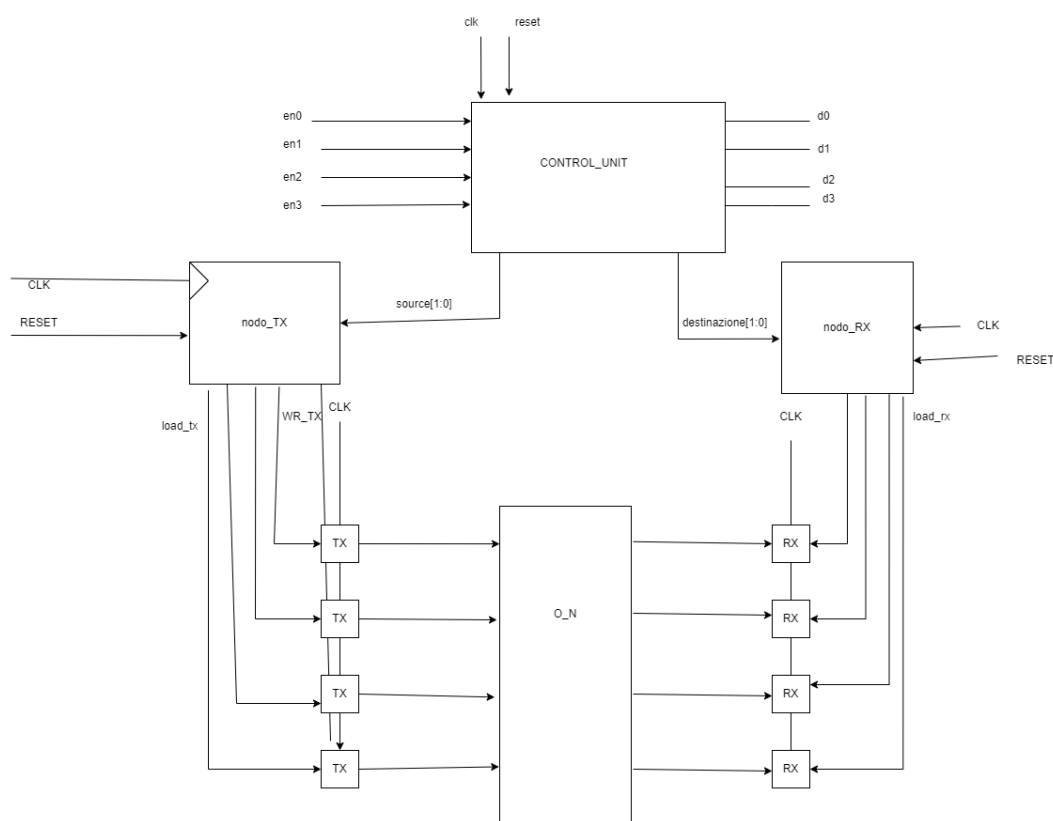


Figura 7.7: Omega Network con Handshaking (Schema a blocchi)

Di seguito sono riportati gli automi per trasmettitore e ricevitore che mostrano la logica legata allo scambio dei messaggi:

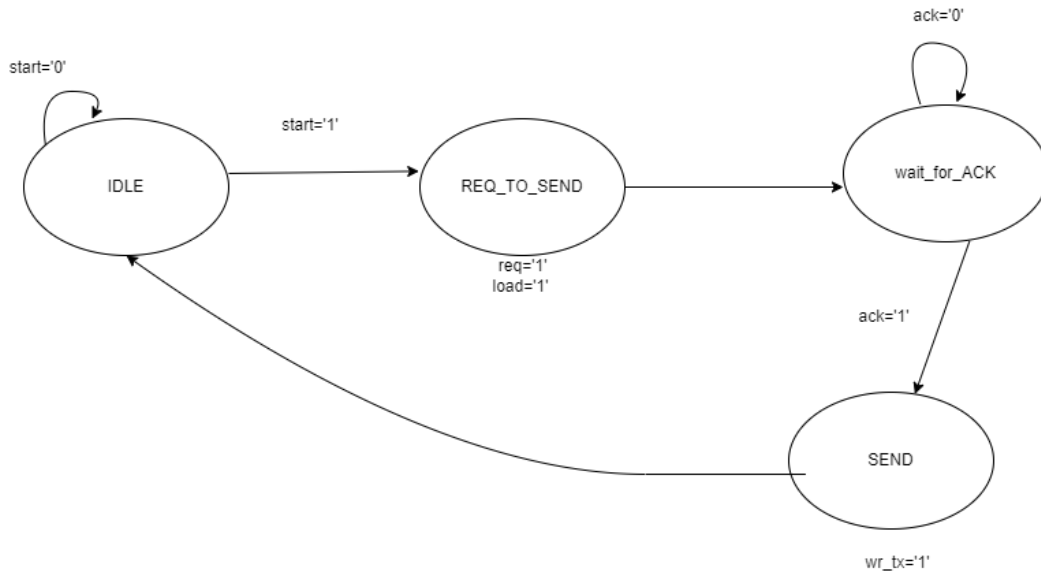


Figura 7.8: Automa Nodo Trasmettitore

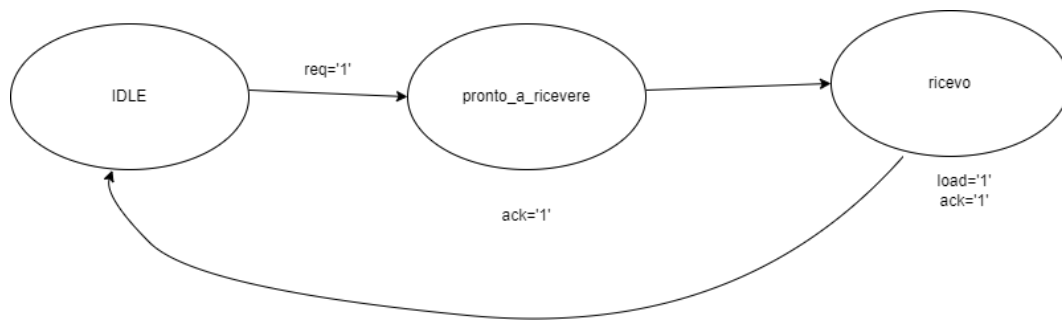


Figura 7.9: Automa Nodo Ricevitore

Implementazione

Implementazione Control Unit Nodo Tx:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6  entity Control_Unit_A is

```

```
7  Port (
8      clock,reset,start : in std_logic;
9      req: out std_logic;
10     ack: in std_logic;
11     load,write: out std_logic
12
13 );
14 end Control_Unit_A;
15
16 architecture Behavioral of Control_Unit_A is
17
18     type state is (IDLE,REQ_TO_SEND,WAIT_FOR_ACK,SEND);
19     signal current_state,next_state: state;
20
21 begin
22     reg_stato: process(clock,reset)
23     begin
24         if(clock'event and clock='1') then
25             if(reset='1') then
26                 current_state <=idle;
27             else
28                 current_state <=next_state;
29             end if;
30         end if;
31     end process;
32     comb: process(current_state,start,ack)
33 begin
34
```

```
35     req<='0';
36     load<='0';
37     write<='0';
38
39
40
41
42 CASE current_state is
43
44     WHEN IDLE =>
45
46
47         if(start='1') then
48             next_state <= REQ_TO_SEND;
49
50         else
51             next_state <= IDLE;
52         end if;
53
54     --fornisce i segnali di caricamento operandi
55     WHEN REQ_TO_SEND=>
56
57         req<='1';
58         load<='1';
59
60         next_state <= WAIT_FOR_ACK;
61
62
```

```
63      WHEN WAIT_FOR_ACK=>
64
65          if ack='0' then
66              next_state<=WAIT_FOR_ACK;
67          else
68              next_state<= SEND;
69          end if;
70
71      WHEN SEND=>
72          write<='1';
73          next_state<= IDLE;
74
75
76
77
78      end CASE;
79  end process;
80 end Behavioral;
```

Implementazione Nodo Tx:

```
1      library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity Nodo_TX is
5      Port (
6          clock, reset, start: in std_logic;
7          src: in std_logic_vector (1 downto 0);
8          ack: in std_logic;
9          load, write: out std_logic_vector(3 downto 0);
```

```
10     req: out std_logic
11
12 );
13 end Nodo_TX;
14
15 architecture Behavioral of Nodo_TX is
16
17 component Control_Unit_A is
18     Port (
19         clock,reset,start : in std_logic;
20         req: out std_logic;
21         ack: in std_logic;
22         load,write: out std_logic
23
24     );
25 end component;
26
27 component demux_1_4 is
28     port(
29         a_dem: in STD_LOGIC;
30         sel_dem: in STD_LOGIC_VECTOR(1 downto 0);
31         Z: out STD_LOGIC_VECTOR (3 downto 0)
32
33     );
34 end component;
35
36 signal l,w: std_logic:='0';
37
```

```
38 begin
39
40 CU: Control_Unit_A port map(clock,reset,start,req,ack,l,w);
41
42 DEMUX_load: demux_1_4 port map(l,src,load);
43 DEMUX_wr:demux_1_4 port map(w,src,write);
44
45
46 end Behavioral;
```

Implementazione Control Unit Nodo Rx:

```
1      library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6  entity Control_Unit_B is
7      Port (
8          clock,reset : in std_logic;
9          req: in std_logic;
10         ack: out std_logic;
11         load: out std_logic
12
13     );
14  end Control_Unit_B;
15
16  architecture Behavioral of Control_Unit_B is
17
18  type state is (IDLE,PRONTO_A_RICEVERE,RICEVO);
```

```
19 signal current_state,next_state: state;
20
21 begin
22 reg_stato: process(clock,reset)
23     begin
24         if(clock'event and clock='1') then
25             if(reset='1') then
26                 current_state <=idle;
27             else
28                 current_state <=next_state;
29             end if;
30         end if;
31     end process;
32 comb: process(current_state,req)
33 begin
34
35     ack<='0';
36     load<='0';
37
38
39
40
41
42 CASE current_state is
43
44     WHEN IDLE =>
```

```

47         if (req='1') then
48
49             next_state <= PRONTO_A_RICEVERE;
50
51         else
52             next_state <= IDLE;
53         end if;
54
55         --fornisce i segnali di caricamento operandi
56         WHEN PRONTO_A_RICEVERE=>
57
58             ack<='1';
59
60
61             next_state <= RICEVO;
62
63
64         WHEN RICEVO=>
65             load<='1';
66             ack<='1';
67
68             next_state<=IDLE;
69
70         end CASE;
71     end process;
72 end Behavioral;

```

Implementazione Nodo RX:

```

1     library IEEE;

```

```
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity Nodo_RX is
5     Port (
6         clock, reset: in std_logic;
7         dst: in std_logic_vector (1 downto 0);
8         ack: out std_logic;
9         load: out std_logic_vector (3 downto 0);
10        req: in std_logic
11    );
12
13 end Nodo_RX;
14
15 architecture Behavioral of Nodo_RX is
16
17     component Control_Unit_B is
18         Port (
19             clock, reset : in std_logic;
20             req: in std_logic;
21             ack: out std_logic;
22             load: out std_logic
23         );
24     end component;
25
26
27     component demux_1_4 is
28         port (
29             a_dem: in STD_LOGIC;
```

```
30     sel_dem: in STD_LOGIC_VECTOR(1 downto 0);
31     Z: out STD_LOGIC_VECTOR (3 downto 0)
32
33 );
34 end component;
35
36 signal l: std_logic:='0';
37
38 begin
39
40 CU: Control_Unit_B port map(clock, reset, req, ack, l);
41
42 DEMUX_load: demux_1_4 port map(l, dst, load);
43
44 end Behavioral;
```

Implementazione Top Module:

```
1     library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5 entity Top_Module is
6 Port (
7
8     clock_A, clock_B, rst, START: in std_logic;
9     i_0, i_1, i_2, i_3: in std_logic;
10    d_0, d_1, d_2, d_3 : in std_logic_vector(1 downto 0);
11    en_0, en_1, en_2, en_3: in std_logic;
12    q0, q1, q2, q3: out std_logic
```

```
13
14
15
16
17 );
18 end Top_Module;
19
20 architecture Behavioral of Top_Module is
21
22
23 signal dst,src : std_logic_vector(1 downto 0):=(others=>'0');
24
25 component control_unit is
26     Port (
27         en0:in std_logic ;
28         en1:in std_logic ;
29         en2:in std_logic ;
30         en3:in std_logic ;
31         d0,d1,d2,d3: in std_logic_vector (1 downto 0);
32         s: out std_logic_vector(1 downto 0);
33         d:out std_logic_vector (1 downto 0)
34     );
35 end component;
36
37
38 component Nodo_TX is
39     Port (
40         clock, reset,start:in std_logic;
```

```
41     src:in std_logic_vector (1 downto 0);
42     ack: in std_logic;
43     load, write: out std_logic_vector(3 downto 0);
44     req: out std_logic
45
46 );
47 end component;
48
49 component reg_wr_1 is
50
51 port (
52     clock: in std_logic;
53     reset: in std_logic;
54     load: in std_logic;
55     write: in std_logic;
56     data_input: in std_logic;
57     data_output: out std_logic
58 );
59 end component;
60
61 component Nodo_RX is
62     Port (
63         clock, reset:in std_logic;
64         dst:in std_logic_vector (1 downto 0);
65         ack: out std_logic;
66         load: out std_logic_vector(3 downto 0);
67         req: in std_logic
68
```

```
69     );
70 end component;
71
72 component registro is
73     Port (
74         clk      : in STD_LOGIC;
75         rst      : in STD_LOGIC;
76         load     : in STD_LOGIC;
77         data     : in STD_LOGIC;
78         q        : out STD_LOGIC
79     );
80 end component;
81
82
83 component omega_network is
84     Port (
85         ingresso0: in std_logic;
86         ingresso1: in std_logic;
87         ingresso2: in std_logic;
88         ingresso3: in std_logic;
89         s: in std_logic_vector(1 downto 0);
90         d: in std_logic_vector(1 downto 0);
91         uscita0: out std_logic;
92         uscita1: out std_logic;
93         uscita2: out std_logic;
94         uscita3: out std_logic
95     );
96
```

```

97 end component;
98
99 signal s,d: std_logic_vector(1 downto 0):=(others=>'0');
100 signal u0,u1,u2,u3: std_logic:='0';
101 signal ack,req: std_logic:='0';
102 signal load_T,load_R,write: std_logic_vector(3 downto 0):=(
    others=>'0');
103 signal i0,i1,i2,i3: std_logic:='0';
104 begin
105
106 CU: control_unit port map(en_0,en_1,en_2,en_3,d_0,d_1,d_2,d_3,
    s,d);
107 --Trasmissione
108 TX: Nodo_TX port map(clock_A,rst,START,s,ack,load_T,write,req)
    ;
109 TX_0 :reg_wr_l port map(clock_A,rst,load_T(0),write(0),i_0,i0)
    ;
110 TX_1 :reg_wr_l port map(clock_A,rst,load_T(1),write(1),i_1,i1)
    ;
111 TX_2 :reg_wr_l port map(clock_A,rst,load_T(2),write(2),i_2,i2)
    ;
112 TX_3 :reg_wr_l port map(clock_A,rst,load_T(3),write(3),i_3,i3)
    ;
113
114 --Ricezione
115 RX: Nodo_RX port map(clock_B,rst,d,ack,load_R,req);
116 reg_0: registro port map(clock_B,rst,load_R(0),u0,q0);
117 reg_1: registro port map(clock_B,rst,load_R(1),u1,q1);

```

```
118 reg_2: registro port map(clock_B,rst,load_R(2),u2,q2);
119 reg_3: registro port map(clock_B,rst,load_R(3),u3,q3);
120
121 O_N:omega_network port map(i0,i1,i2,i3,s,d,u0,u1,u2,u3);
122
123
124 end Behavioral;
```

Simulazione

Di seguito è riportata la simulazione dell'omega network:

```
1      --
      -----
2  -- Company:
3  -- Engineer:
4  --
5  -- Create Date: 03.09.2024 18:56:16
6  -- Design Name:
7  -- Module Name: TB - Behavioral
8  -- Project Name:
9  -- Target Devices:
10 -- Tool Versions:
11 -- Description:
12 --
13 -- Dependencies:
14 --
15 -- Revision:
```

```
16 -- Revision 0.01 - File Created
17 -- Additional Comments:
18 --
19 --
    -----
20
21
22 library IEEE;
23 use IEEE.STD_LOGIC_1164.ALL;
24
25
26 entity TB is
27 -- Port ( );
28 end TB;
29
30 architecture Behavioral of TB is
31
32
33 component
34 Top_Module is
35 Port (
36
37     clock_A, clock_B, rst, START: in std_logic;
38     i_0, i_1, i_2, i_3: in std_logic;
39     d_0, d_1, d_2, d_3 : in std_logic_vector(1 downto 0);
40     en_0, en_1, en_2, en_3: in std_logic;
41     q0, q1, q2, q3: out std_logic
```

```
42
43
44
45
46 );
47 end component;
48
49 signal clock_0, clock_1, rst, START, i0, i1, i2, i3, e0, e1, e2, e3,
    uscita0, uscita1, uscita2, uscita3: std_logic := '0';
50 signal d1, d2, d3, d0 : std_logic_vector(1 downto 0) := (others
    => '0');
51
52 constant clock_period_A: time := 10 ns;
53 constant clock_period_B: time := 11 ns;
54
55
56
57
58 begin
59 CLK_process_A: process
60 begin
61     clock_0 <= '0';
62     wait for clock_period_A/2;
63     clock_0 <= '1';
64     wait for clock_period_A/2;
65 end process;
66
67
```

```
68 CLK_process_B: process
69 begin
70     clock_1<='0';
71     wait for clock_period_B/2;
72     clock_1<='1';
73     wait for clock_period_B/2;
74 end process;
75
76 UUT: Top_module port map( clock_0,clock_1,rst,START,i0,i1,i2,
77     i3,d0,d1,d2,d3,e0,e1,e2,e3,uscita0,uscita1,uscita2,uscita3);
78
79
80 stim_proc: process
81 begin
82
83
84 rst<='1';
85 wait for 10 ns;
86 rst<='0';
87 START<='1';
88 i1<='1';
89 i0<='0';
90 i2<='0';
91 i3<='0';
92
93 d1<="11";
94 d0<="00";
```

```

95 d2<="01";
96 d3<="10";
97
98 e1<='1';
99 e0<='0';
100 e2<='0';
101 e3<='0';
102
103 wait for 30 ns;
104 START<='0';
105
106
107 wait ;
108 end process;
109
110 end Behavioral;

```

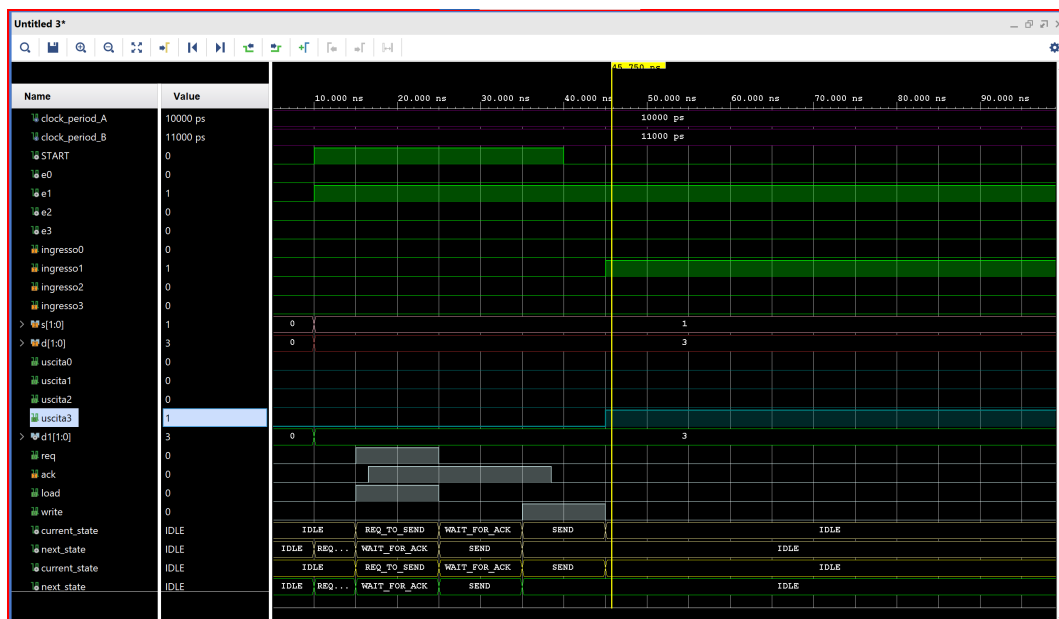


Figura 7.10: Simulazione Omega Network

Capitolo 8

Esercizio 12

Progettare, implementare in VHDL e testare mediante simulazione (e, opzionalmente, mediante sintesi su board), un sistema per l'implementazione di un cifrario PRESENT, le cui specifiche sono riportate nel documento allegato.

Progettare, implementare in VHDL e testare mediante simulazione (e, opzionalmente, mediante sintesi su board), un sistema le cui specifiche siano definite dallo studente e rientrino in una delle seguenti tipologie:

- Modifica di esercizi già proposti (processore, rete di interconnessione o interfaccia seriale) mediante aggiunta/aggiornamento di funzionalità.
- Progetto di sistemi che assolvono a specifici compiti noti.
- Progetto di sistemi che integrano opportunamente componenti visti a lezione (contatori, registri, macchine aritmetiche, ecc.).

8.0.1 Progetto e Architettura

Per tale esercizio si è scelto di modificare l'esercizio 8, "Comunicazione con Handshaking", aggiungendo un cifrario da noi ideato. L'utente, tramite gli switch della board Nexys A7, potrà inserire delle stringhe di bit, che verranno memorizzate dal nodo A in un apposito registro e successivamente la stringa verrà inviata al nodo B contenente il blocco di conversione, in grado di tradurla in caratteri visualizzabili sul display.

Per garantire il corretto funzionamento del sistema, l'architettura è stata modificata nel seguente modo:

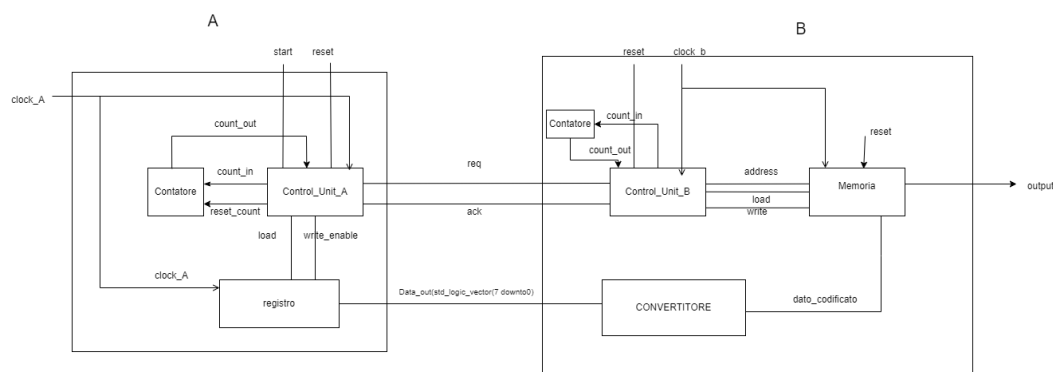


Figura 8.1: Architettura Esercizio 12

Di seguito riportiamo gli automi dei nodi A e B.

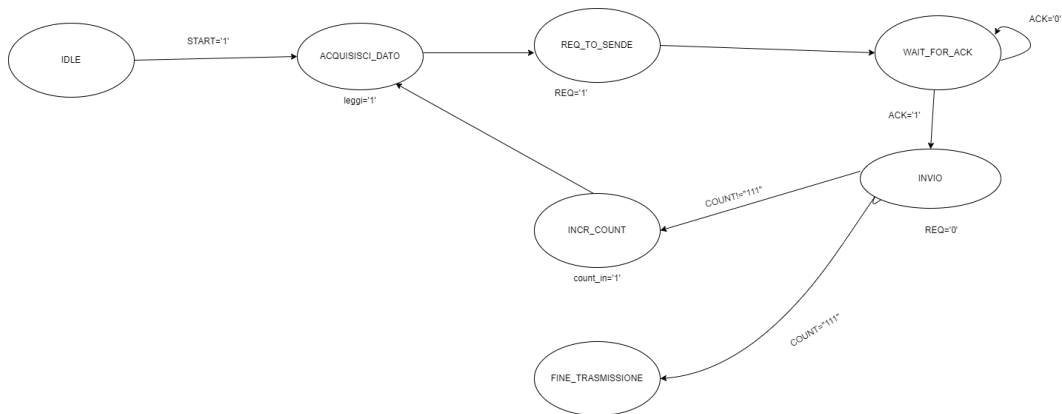


Figura 8.2: Automa Nodo A

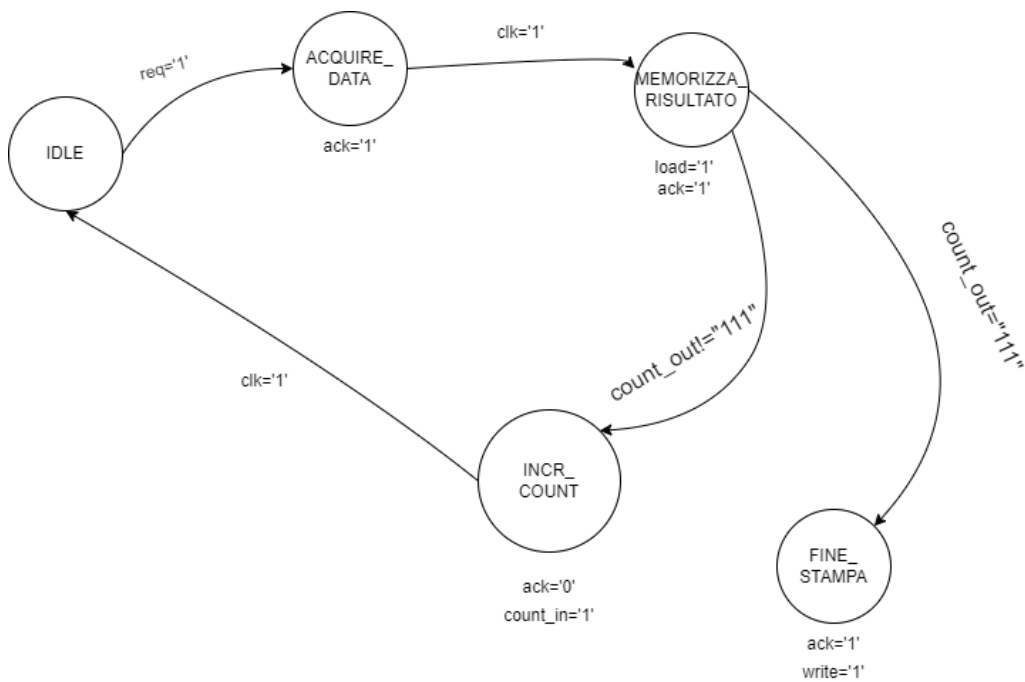


Figura 8.3: Automa Nodo B

8.0.2 Implementazione

Implementazioen Control Unit Nodo A:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3

```

```
4
5 entity Control_Unit_A is
6   Generic(
7       Log: integer:=2
8   );
9
10  Port (
11      clk_a: in std_logic;
12      reset: in std_logic;
13      start: in std_logic;
14      acquire: in std_logic;
15      ack: in std_logic;
16      count_out: in std_logic_vector(Log-1 downto 0);
17      count_in : out std_logic;
18      reset_count: out std_logic;
19      leggi: out std_logic;
20      req:out std_logic;
21      led: out std_logic
22  );
23  end Control_Unit_A;
24
25  architecture Behavioral of Control_Unit_A is
26
27
28
29  type state is (IDLE,ACQUISISCI_DATO,REQ_TO_SEND,WAIT_FOR_ACK,
30      INVIO,INCR_COUNT,FINE_TRASMISSIONE);
```

```
31
32
33 begin
34
35
36
37 reg_stato: process (clk_A, reset)
38     begin
39         if (clk_a'event and clk_a='1') then
40             if (reset='1') then
41                 reset_count<='1';
42                 current_state <=idle;
43             else
44                 current_state <=next_state;
45             end if;
46         end if;
47     end process;
48
49 comb: process (current_state, start, count_out, ack, acquire)
50 begin
51
52     leggi<='0';
53     count_in<='0';
54     led<='0';
55     req<='0';
56     reset_count<='0';
57
58 CASE current_state is
```

```
59
60     WHEN IDLE =>
61
62
63         if(start='1') then
64             next_state <= ACQUISISCI_DATO;
65
66         else
67             next_state <= IDLE;
68         end if;
69
70     --fornisce i segnali di caricamento operandi
71
72     WHEN ACQUISISCI_DATO=>
73         led<='1';
74         if acquire ='1' then
75             leggi<='1';
76             next_state<=REQ_TO_SEND;
77         else
78             next_state<= ACQUISISCI_DATO;
79         end if;
80
81
82     WHEN REQ_TO_SEND=>
83
84
85         req<='1';
86
```

```
87         next_state <= WAIT_FOR_ACK;
88
89         --acquisisce gli operandi, su cui il sommatore inizia
90         a lavorare immediatamente
91
92     WHEN WAIT_FOR_ACK=>
93
94         req<='1';
95         if ack='1' then
96             next_state<=INVIO;
97         else
98             next_state<=WAIT_FOR_ACK;
99         end if;
100
101     WHEN INVIO=>
102         req<='0';
103
104         if count_out="111" then
105
106             next_state<= FINE_TRASMISSIONE;
107         else
108
109             next_state<=INCR_COUNT;
110         end if;
111
112
113
```

```
114         WHEN INCR_COUNT =>
115             count_in<='1';
116
117             --if ack<='0' then
118                 next_state<=ACQUISISCI_DATO;
119             --end if;
120
121
122         WHEN FINE TRASMISSIONE =>
123             reset_count<='1';
124
125             next_state<= FINE TRASMISSIONE;
126
127
128
129
130     end CASE;
131 end process;
132
133
134 end Behavioral;
```

Implementazione Nodo A:

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5
6 entity A is
```

```
7
8 Port (
9     clk_a: in std_logic;
10    reset: in std_logic;
11    START: in std_logic;
12    ACQUIRE: in std_logic;
13    data_in : in std_logic_vector(7 downto 0);
14    ack: in std_logic;
15    req:out std_logic;
16    data_out:out std_logic_vector(7 downto 0);
17    led_ACQUIRE: out std_logic
18
19    );
20 end A;
21
22 architecture Behavioral of A is
23
24
25 component Control_Unit_A is
26 Generic(
27     Log: integer:=2
28 );
29
30 Port (
31     clk_a: in std_logic;
32     reset: in std_logic;
33     start: in std_logic;
34     acquire: in std_logic;
```

```
35     ack: in std_logic;
36     count_out: in std_logic_vector(Log-1 downto 0);
37     count_in : out std_logic;
38     reset_count: out std_logic;
39     leggi: out std_logic;
40     req:out std_logic;
41     led: out std_logic
42 );
43 end component;
44
45 component registro_8 is
46     port (
47         A          : in std_logic_vector(7 downto 0);
48         clk, rst    : in std_logic;
49         load        : in std_logic;
50         write_enable: in std_logic;
51         data_out     : out std_logic_vector(7 downto 0)
52     );
53 end component;
54
55 --contatore
56 component cont_mod8 is
57     generic (
58         Log: integer := 2
59     );
60
61     port( clock, reset: in std_logic;
62          count_in: in std_logic;
```

```

63         count: out std_logic_vector(Log-1 downto 0));
64 end component;
65
66
67
68
69 signal count_out: std_logic_vector(2 downto 0) := (others => '0');
70 signal count_in, reset_count, leggi, scrivi : std_logic := '0';
71 begin
72
73 --Control _unit
74 CU: Control_Unit_A generic map(3) port map(clk_a, reset, START,
       ACQUIRE, ack, count_out, count_in, reset_count, leggi, req,
       led_ACQUIRE);
75 --Contatore
76 Contatore: cont_mod8 generic map(3) port map(clk_a, reset,
       count_in, count_out);
77 -- Registro
78 Reg_1: registro_8 port map(data_in, clk_a, reset, leggi, scrivi,
       data_out);
79
80 end Behavioral;

```

Implementazione Control Unit B:

```

1     library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5 entity Control_Unit_B is

```

```
6  Port (
7      clock: in std_logic;
8      reset: in std_logic;
9      count_out : in std_logic_vector(2 downto 0);
10     req: in std_logic;
11     ack: out std_logic;
12     count_in :out std_logic;
13     load,write: out std_logic;
14     address: out std_logic_vector(2 downto 0)
15
16     );
17 end Control_Unit_B;
18
19 architecture Behavioral of Control_Unit_B is
20
21     type state is (IDLE,ACQUIRE_DATA,MEMORIZZA_RISULTATO,
22                    INCR_COUNT,FINE_STAMPA);
23
24     signal current_state,next_state: state;
25
26
27 begin
28
29     reg_stato: process(clock,reset)
30
31         begin
32             if(clock'event and clock='1') then
33                 if(reset='1') then
34
35                     current_state <=idle;
```

```
33         else
34             current_state <=next_state;
35         end if;
36     end if;
37 end process;
38
39 comb: process(current_state, req, count_out)
40 begin
41
42     load<='0';
43     write<='0';
44     ack<='0';
45     count_in<='0';
46     address<=count_out;
47
48
49 CASE current_state is
50
51     WHEN IDLE =>
52
53
54         if(req='1') then
55             next_state <= ACQUIRE_DATA;
56
57         else
58             next_state <= IDLE;
59         end if;
60
```

```
61      --fornisce i segnali di caricamento operandi
62      WHEN ACQUIRE_DATA=>
63
64
65          ack<='1';
66          next_state <= MEMORIZZA_RISULTATO;
67
68      --acquisisce gli operandi, su cui il sommatore inizia
69      --a lavorare immediatamente
70      WHEN MEMORIZZA_RISULTATO=>
71
72          load<='1';
73          ack<='1';
74          if count_out="111" THEN
75              next_state<=FINE_STAMPA;
76          else
77              next_state<= INCR_COUNT;
78          end if;
79
80      WHEN INCR_COUNT =>
81
82          count_in<='1';
83          ack<='0';
84
85          next_state<= IDLE;
86
87
```

```
88
89
90     WHEN FINE_STAMPA=>
91
92         ack<='1';
93         write<='1';
94         next_state<= FINE_STAMPA;
95
96
97     end CASE;
98 end process;
99
100
101 end Behavioral;
```

Implementazione Nodo B:

```
1     library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4
5 entity B is
6 generic (
7     number_of_bit:integer:=4;
8     Log: integer:=2
9 );
10
11 Port (
12     clk_b: in std_logic;
13     reset: in std_logic;
```

```
14     data_in : in std_logic_vector(7 downto 0);
15     ack: out std_logic;
16     req:in std_logic;
17     data_out:out std_logic_vector(39 downto 0)
18
19     );
20 end B;
21
22 architecture Behavioral of B is
23
24
25 component Control_Unit_B is
26     Port (
27         clock: in std_logic;
28         reset: in std_logic;
29         count_out : in std_logic_vector(2 downto 0);
30         req: in std_logic;
31         ack: out std_logic;
32         count_in :out std_logic;
33         load,write: out std_logic;
34         address: out std_logic_vector(2 downto 0)
35
36     );
37
38 end component;
39 --Memoria
40 component mem is
41 generic(
```

```
42     lunghezza_indirizzo: integer := 3;
43     lunghezza_dati: integer := 4;
44     number_of_location: integer:= 8
45
46 );
47 port (
48     clock: in std_logic;
49     reset: in std_logic;
50     read: in std_logic;
51     write: in std_logic;
52     address: in std_logic_vector((lunghezza_indirizzo - 1)
53         downto 0);
54     data_in: in std_logic_vector((lunghezza_dati- 1) downto 0)
55         ;
56     data_output: out std_logic_vector (39 downto 0)
57 );
58
59 --contatore
60 component cont_mod8 is
61     generic (
62         Log: integer := 3
63     );
64
65     port( clock, reset: in std_logic;
66         count_in: in std_logic;
67         count: out std_logic_vector(Log-1 downto 0));
```

```
68 end component;
69
70 --CONVERTITORE
71 component blocco_conversione is
72     port (
73         input : in std_logic_vector(7 downto 0);
74         encoded_output : out std_logic_vector(4 downto 0)
75     );
76 end component;
77
78
79
80 signal count_out,address: std_logic_vector(2 downto 0):=(
81     others=>'0');
82 signal count_in,reset_count,load_M,write_M : std_logic:='0';
83 signal dato_codificato: std_logic_vector(4 downto 0):=(others
84     =>'0');
85
86 begin
87
88 --Control _unit
89 CU_B: Control_Unit_B port map(clk_b,reset,count_out,req,ack,
90     count_in,load_M,write_M,address);
91
92 --Contatore
93 Contatore: cont_mod8 generic map(3) port map(clk_b,reset,
94     count_in,count_out);
95
96 CONVERTITORE:blocco_conversione port map(data_in,
97     dato_codificato);
98
99 -- mem
```

```

91 MEMORIA: mem generic map(3,5,8) port map( clk_b,reset,load_M,
    write_M,address,dato_codificato,data_out);
92
93
94 end Behavioral;

```

8.0.3 Simulazione

Implementazione Blocco Convertitore:

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity blocco_conversione is
6      port (
7          input : in std_logic_vector(7 downto 0);
8          encoded_output : out std_logic_vector(4 downto 0)
9      );
10 end blocco_conversione;
11
12 architecture behavioral of blocco_conversione is
13 begin
14     process(input)
15     begin
16         case input is
17             when "00000000" =>
18                 --decimal_output <= 0;
19                 encoded_output <= "00000";

```

```
20      when "00000001" =>
21          --decimal_output <= 1;
22          encoded_output <= "00001";
23      when "00000010" =>
24          --decimal_output <= 2;
25          encoded_output <= "00010";
26      when "00000011" =>
27          --decimal_output <= 3;
28          encoded_output <= "00011";
29      when "00000100" =>
30          --decimal_output <= 4;
31          encoded_output <= "00100";
32      when "00000101" =>
33          --decimal_output <= 5;
34          encoded_output <= "00101";
35      when "00000110" =>
36          --decimal_output <= 6;
37          encoded_output <= "00110";
38      when "00000111" =>
39          --decimal_output <= 7;
40          encoded_output <= "00111";
41      when "00001000" =>
42          --decimal_output <= 8;
43          encoded_output <= "01000";
44      when "00001001" =>
45          --decimal_output <= 9;
46          encoded_output <= "01001";
47      when "01000001" =>
```

```
48      --ASCII<=A;
49      encoded_output <= "01010";
50  when "01000010" =>
51      --ASCII<=B;
52      encoded_output <= "01011";
53  when "01000011" =>
54      --ASCII<=C;
55      encoded_output <= "01100";
56  when "01000100" =>
57      --ASCII<=D;
58      encoded_output <= "01101";
59  when "01000101" =>
60      --ASCII<= E;
61      encoded_output <= "01110";
62  when "01000110" =>
63      --ASCII <=F;
64      encoded_output <= "01111";
65
66  when "01000111" =>
67      --ASCII <=G
68      encoded_output<= "10000";
69  when "01001000" =>
70      --ASCII<=H
71      encoded_output<= "10001";
72  when "01001001" =>
73      --ASCII<=I
74      encoded_output<= "10010";
75  when "01001100" =>
```

```
76         --ASCII<= L
77         encoded_output<= "10011";
78     when "01001110" =>
79         --ASCII<= N
80         encoded_output<= "10100";
81     when "01001111" =>
82         --ASCII<= O
83         encoded_output<= "10101";
84     when "01010000" =>
85         --ASCII<= P
86         encoded_output<= "10110";
87     when "01010001" =>
88         --ASCII<=Q
89         encoded_output<= "10111";
90     when "01010010" =>
91         --ASCII<=R
92         encoded_output<= "11000";
93     when "01010011" =>
94         --ASCII<= S
95         encoded_output<= "11001";
96     when "01010100" =>
97         --ASCII<= T
98         encoded_output<= "11010";
99     when "01010101" =>
100         --ASCII<= U
101         encoded_output<= "11011";
102     when "01011001" =>
103         --ASCII<= Y
```

```
104         encoded_output<= "11100";
105     when others =>
106         --tradurro uno spazio/trattino
107         encoded_output<= "11101";
108     end case;
109 end process;
110 end behavioral;
```

Implementazione Top Module:

```
1      --
2      -----
3
4
5  -- Company:
6  -- Engineer:
7  --
8  -- Create Date: 22.04.2024 19:50:05
9  -- Design Name:
10 -- Module Name: Top_module - Behavioral
11 -- Project Name:
12 -- Target Devices:
13 -- Tool Versions:
14 -- Description:
15 --
16 -- Dependencies:
17 --
18 -- Revision:
19 -- Revision 0.01 - File Created
20 -- Additional Comments:
21 --
```

```
19  --
20
21
22  library IEEE;
23  use IEEE.STD_LOGIC_1164.ALL;
24
25
26  entity Top_module is
27      Port (
28          CLOCK: in std_logic;
29          RESET: in std_logic;
30          START: in std_logic;
31          ACQUIRE: in std_logic;
32          switch: in std_logic_vector(7 downto 0);
33          cathodes_out: out std_logic_vector(7 downto 0);
34          anodes_out: out std_logic_vector(7 downto 0);
35          led: out std_logic
36
37      );
38
39  end Top_module;
40
41  architecture Behavioral of Top_module is
42      --NODO A
43      component A is
44
```

```
45
46 Port (
47     clk_a: in std_logic;
48     reset: in std_logic;
49     START: in std_logic;
50     ACQUIRE: in std_logic;
51     data_in : in std_logic_vector(7 downto 0);
52     ack: in std_logic;
53     req:out std_logic;
54     data_out:out std_logic_vector(7 downto 0);
55     led_ACQUIRE: out std_logic
56
57 );
58 end component;
59
60
61 --NODO B
62 component B is
63 generic (
64     number_of_bit:integer:=4;
65     Log: integer:=2
66 );
67
68 Port (
69     clk_b: in std_logic;
70     reset: in std_logic;
71     data_in : in std_logic_vector(7 downto 0);
72     ack: out std_logic;
```

```
73     req:in std_logic;
74     data_out:out std_logic_vector(39 downto 0)
75
76     );
77 end component;
78
79 --CLOCK DIVIDER
80 component clock_divider is
81     generic(
82         clock_frequency_in : integer := 100000000;--
83         100MHz
84         clock_frequency_out : integer := 500 --500 Hz
85     );
86     Port ( clock_in : in  STD_LOGIC;
87           reset : in  STD_LOGIC;
88           clock_out : out  STD_LOGIC);
89 end component;
90
91 --display
92 component display_seven_segments is
93     Generic(
94         CLKIN_freq : integer := 100000000;
95         CLKOUT_freq : integer := 500
96     );
97     Port ( CLK : in  STD_LOGIC;
98           RST : in  STD_LOGIC;
99           VALUE : in  STD_LOGIC_VECTOR (39 downto 0);
100          ENABLE : in  STD_LOGIC_VECTOR (7 downto 0); --
```

```

        decide quali cifre abilitare
100     DOTS : in  STD_LOGIC_VECTOR (7 downto 0); -- decide
        quali punti visualizzare
101     ANODES : out  STD_LOGIC_VECTOR (7 downto 0);
102     CATHODES : out  STD_LOGIC_VECTOR (7 downto 0));
103 end component;
104 --button debouncer
105 component ButtonDebouncer is
106     generic (
107         CLK_period: integer := 10; -- periodo del clock
        della board 10 nanosecondi
108         btn_noise_time: integer := 6500000 --intervallo di
        tempo in cui si ha l'oscillazione del bottone
109                                     --assumo che duri 6.5
        ms=6500microsec
        =6500000ns
110     );
111     Port ( RST : in STD_LOGIC;
112           CLK : in STD_LOGIC;
113           BTN : in STD_LOGIC;
114           CLEARED_BTN : out STD_LOGIC);
115 end component;
116
117 signal clock_diviso, clock_diviso_B: std_logic:='0';
118 signal ack, req, cleared_start: std_logic:='0';
119 signal dato_trasmesso: std_logic_vector(7 downto 0):=(others
    =>'0');
120 signal dato_out: std_logic_vector(39 downto 0):=(others=>'0');

```

```
121
122 begin
123
124 --button_debouncer: ButtonDebounce generic map(10,6500000)
    port map(RESET,CLOCK,START,cleared_start);
125
126 divisore_di_frequenza: clock_divider generic map(100000000,1)
    port map(CLOCK,RESET,clock_diviso);
127
128 NODO_A:A port map(clock_diviso,RESET,START,ACQUIRE,switch,ack,
    req,dato_trasmesso,led);
129
130 divisore_di_frequenza_B: clock_divider generic map
    (100000000,1) port map(CLOCK,RESET,clock_diviso_B);
131
132 NODO_B : B generic map (4,2) port map(clock_diviso_B,RESET,
    dato_trasmesso,ack,req,dato_out);
133
134
135 Display_a_sette_segmenti: display_seven_segments generic map
    (100000000,500)
136 port map(CLK=>CLOCK,RST=>RESET,VALUE=>dato_out,ENABLE=>"
    11111111",DOTS=>"00000000",ANODES=>anodes_out,CATHODES=>
    cathodes_out);
137
138
139
140
```

```
end Behavioral;
```

Di seguito mostriamo un esempio di output sulla board, ottenuto alzando gli opportuni switch:

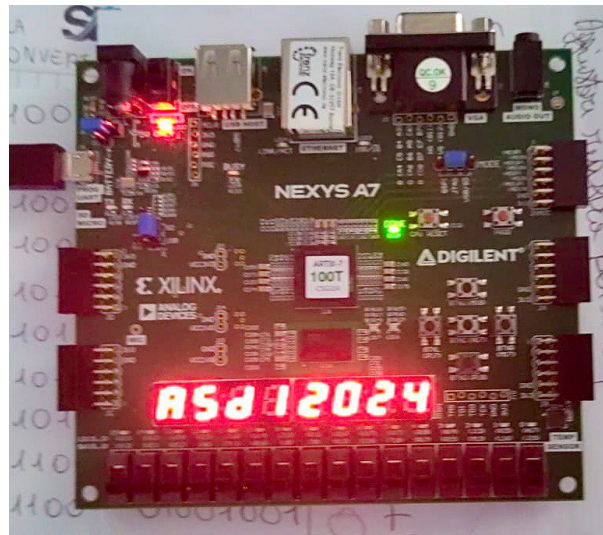


Figura 8.4: Output Simulazione

Capitolo 9

Timing Analysis

Per completare l'elaborato abbiamo deciso di effettuare la **Timing Analysis** dell'esercizio 7 (Moltiplicatore di Booth).

La timing analysis è un processo che ci permette di produrre un report più o meno accurato riguardo i vincoli temporali del design.

L'obiettivo è quello di verificare qual è la frequenza massima dell'implementazione del moltiplicatore di Booth.

L'approccio per questo tipo di verifica è quello di utilizzare periodi di clock decrementati progressivamente al fine di trovare uno `slack` negativo, differenza tra il tempo richiesto e il tempo di arrivo sulle uscite. Come prima cosa abbiamo modificato il constraint "*Nexys-A7*" in *Vivado* nel seguente modo:

```
### Clock signal
set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { CLOCK }];
create_clock -add -name sys_clk_pin -period 10 -waveform {0 5.00} [get_ports { CLOCK }
```

Figura 9.1: Modifica constraint

Di seguito sono riportati tutti i risultati relativi alle variazioni del periodo del Clock:

Clock 10 ns

```
6: ### Clock signal
7: set_property -dict ( PACKAGE_PIN E3 IOSTANDARD LVCMOS33 ) [get_ports { CLOCK }];
8: create_clock -add -name sys_clk_pin -period 10 -waveform {0 5.00} [get_ports { CLOCK }];
9:
10:
11: ##Switches
12: set_property -dict ( PACKAGE_PIN J15 IOSTANDARD LVCMOS33 ) [get_ports { switch[0] }];
13: set_property -dict ( PACKAGE_PIN L16 IOSTANDARD LVCMOS33 ) [get_ports { switch[1] }];
14: set_property -dict ( PACKAGE_PIN M13 IOSTANDARD LVCMOS33 ) [get_ports { switch[2] }];
```

Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4,740 ns	Worst Hold Slack (WHS): 0,140 ns	Worst Pulse Width Slack (WPWS): 4,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 67	Total Number of Endpoints: 67	Total Number of Endpoints: 58

All user specified timing constraints are met.

Clock 7 ns

```
6: ### Clock signal
7: set_property -dict ( PACKAGE_PIN E3 IOSTANDARD LVCMOS33 ) [get_ports { CLOCK }];
8: create_clock -add -name sys_clk_pin -period 10 -waveform {0 5.00} [get_ports { CLOCK }];
9:
10:
11: ##Switches
12: set_property -dict ( PACKAGE_PIN J15 IOSTANDARD LVCMOS33 ) [get_ports { switch[0] }];
13: set_property -dict ( PACKAGE_PIN L16 IOSTANDARD LVCMOS33 ) [get_ports { switch[1] }];
14: set_property -dict ( PACKAGE_PIN M13 IOSTANDARD LVCMOS33 ) [get_ports { switch[2] }];
```

Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4,740 ns	Worst Hold Slack (WHS): 0,140 ns	Worst Pulse Width Slack (WPWS): 4,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 67	Total Number of Endpoints: 67	Total Number of Endpoints: 58

All user specified timing constraints are met.

Clock 6 ns

```
4: ## - rename the used ports (in each line, after get_ports) according to the top level
5:
6: ### Clock signal
7: set_property -dict ( PACKAGE_PIN E3 IOSTANDARD LVCMOS33 ) [get_ports { CLOCK }];
8: create_clock -add -name sys_clk_pin -period 6 -waveform {0 3} [get_ports { CLOCK }];
9:
10:
11: ##Switches
12: set_property -dict ( PACKAGE_PIN J15 IOSTANDARD LVCMOS33 ) [get_ports { switch[0] }];
13: set_property -dict ( PACKAGE_PIN L16 IOSTANDARD LVCMOS33 ) [get_ports { switch[1] }];
14: set_property -dict ( PACKAGE_PIN M13 IOSTANDARD LVCMOS33 ) [get_ports { switch[2] }];
```

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0,740 ns	Worst Hold Slack (WHS): 0,140 ns	Worst Pulse Width Slack (WPWS): 2,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 67	Total Number of Endpoints: 67	Total Number of Endpoints: 58

All user specified timing constraints are met.

Clock 5,50 ns

```

5:
6:   ### Clock signal
7:   set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { CLOCK }];
8:   create_clock -add -name sys_clk_pin -period 5.50 -waveform {0 2.75} [get_ports { CLOCK }];
9:
10:
11:  ###Switches
12:  #set_property -dict { PACKAGE_PIN J15      IOSTANDARD LVCMOS33 } [get_ports { switch[0] }];
13:  #set_property -dict { PACKAGE_PIN L16      IOSTANDARD LVCMOS33 } [get_ports { switch[1] }];
14:  #set_property -dict { PACKAGE_PIN M13      IOSTANDARD LVCMOS33 } [get_ports { switch[2] }];

```

Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0,240 ns	Worst Hold Slack (WHS): 0,140 ns	Worst Pulse Width Slack (WPWS): 2,250 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 67	Total Number of Endpoints: 67	Total Number of Endpoints: 58

All user specified timing constraints are met.

Clock 5,25 ns

```

5:
6:   ### Clock signal
7:   set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { CLOCK }];
8:   create_clock -add -name sys_clk_pin -period 5.25 -waveform {0 2.75} [get_ports { CLOCK }];
9:
10:
11:  ###Switches
12:  #set_property -dict { PACKAGE_PIN J15      IOSTANDARD LVCMOS33 } [get_ports { switch[0] }];
13:  #set_property -dict { PACKAGE_PIN L16      IOSTANDARD LVCMOS33 } [get_ports { switch[1] }];
14:  #set_property -dict { PACKAGE_PIN M13      IOSTANDARD LVCMOS33 } [get_ports { switch[2] }];

```

Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0,240 ns	Worst Hold Slack (WHS): 0,140 ns	Worst Pulse Width Slack (WPWS): 2,250 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 67	Total Number of Endpoints: 67	Total Number of Endpoints: 58

All user specified timing constraints are met.

Clock 5 ns

```

5:
6:   ### Clock signal
7:   set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { CLOCK }];
8:   create_clock -add -name sys_clk_pin -period 5.00 -waveform {0 2.50} [get_ports { CLOCK }];
9:
10:
11:  ###Switches
12:  #set_property -dict { PACKAGE_PIN J15      IOSTANDARD LVCMOS33 } [get_ports { switch[0] }];
13:  #set_property -dict { PACKAGE_PIN L16      IOSTANDARD LVCMOS33 } [get_ports { switch[1] }];
14:  #set_property -dict { PACKAGE_PIN M13      IOSTANDARD LVCMOS33 } [get_ports { switch[2] }];

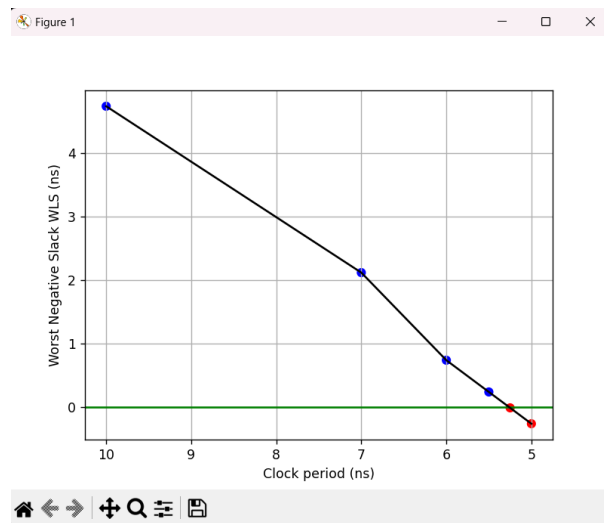
```

Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -0,260 ns	Worst Hold Slack (WHS): 0,140 ns	Worst Pulse Width Slack (WPWS): 2,000 ns
Total Negative Slack (TNS): -0,260 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 1	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 67	Total Number of Endpoints: 67	Total Number of Endpoints: 58

milij constraints are not met.

Di seguito è riportato un grafico con l'andamento dei vari parametri:



Provando diversi clock, i primi valori negativi di slack risultano a partire dai 5,25 ns.

Dunque è opportuno utilizzare il periodo di clock 5,5 ns per calcolare la frequenza massima di funzionamento, in quanto è l'ultimo valore tale per cui lo slack risulta positivo.

$$f_{\text{MAX}} = \frac{1}{T - \text{WNS}} = \frac{1}{5.50 \text{ ns} - 0.24 \text{ ns}} \approx 1.90 \times 10^8 \text{ Hz} = 190 \text{ MHz}$$

Quindi, la frequenza massima di funzionamento del moltiplicatore di Booth sulla board sarà di 190 MHz.